ITA06 – MACHINE LEARNING

Assignment 3 – CO3: Bayesian and Computational Learning

Name: M. Pranay

Roll No.: 192525382

Q1 – Bayes Theorem and Concept Learning for Hypothesis Selection

1.1 Bearing-fault screening with Bayes theorem

Let F = bearing is faulty. P(F) = 0.03, P(+ | F) = 0.95, P(+ | ¬F) = 0.08.

$$P(+) = 0.95\times0.03 + 0.08\times0.97 = 0.0285 + 0.0776 = 0.1061$$

$$P(F \mid +) = 0.0285 / 0.1061 = 0.2686 \ (\approx 26.9\ \%)$$

For two independent positive tests the likelihoods multiply:

$$P(F \mid +,+) = (0.95^2\times0.03) / (0.95^2\times0.03 + 0.08^2\times0.97) = 0.027075 / 0.033283 = 0.8135 \ (\approx 81.3\ \%)$$

Odds form check: prior odds = 0.03/0.97 = 0.0309; likelihood ratio per test = 0.95/0.08 = 11.875. One test: 0.0309×11.875 = 0.367 → p = 0.2686. Two tests: $0.0309\times11.875^2 = 4.36$ → p = 0.8135. Equivalent sequential view: the posterior 0.2686 after the first test becomes the prior for the second.

Effect of the base rate (same test, different prior)

Prior P(F)	P(F | one positive test)
0.001	0.0117
0.03	0.2686
0.1	0.5689
0.3	0.8358
0.5	0.9223

Role of the prior (base rate). The test is "95 % sensitive", yet a single positive means only a 26.9 % chance of a real fault, because faults are rare: of every 1000 bearings, 28.5 are true positives but 77.6 healthy bearings also test positive, so most positives are false alarms. Ignoring the base rate (confusing P(+|F) with P(F|+)) would overstate the risk about 3.5-fold. The prior also tells us when a second test is worth buying: it lifts the belief from 27 % to 81 %.

1.2 Find-S and Candidate-Elimination on D1–D6

Hypotheses are conjunctions <Temperature, Load, Vibration> where each slot is a value, "?" (any) or Ø (none). Initial boundaries: S0 = <Ø,Ø,Ø>, G0 = <?,?,?>. Find-S ignores negative examples.

Find-S trace

Example	Label	Find-S hypothesis
D1	Yes	<High, Heavy, High>
D2	Yes	<High, Heavy, ?>
D3	No	<High, Heavy, ?>
D4	No	<High, Heavy, ?>
D5	Yes	<?, Heavy, ?>
D6	No	<?, Heavy, ?>

Candidate-Elimination trace (S and G after every example)

After	Label	Specific boundary S	General boundary G
D1	+	<High, Heavy, High>	<?, ?, ?>
D2	+	<High, Heavy, ?>	<?, ?, ?>
D3	−	<High, Heavy, ?>	<High, ?, ?>, <?, Heavy, ?>
D4	−	<High, Heavy, ?>	<?, Heavy, ?>
D5	+	<?, Heavy, ?>	<?, Heavy, ?>

After	Label	Specific boundary S	General boundary G
D6	–	<?, Heavy, ?>	<?, Heavy, ?>

- **D1, D2 (positive):** S generalises minimally (<High,Heavy,High> → <High,Heavy,?>); G stays <?,?,?>.
- **D3 (negative: Normal, Light, Low):** G is specialised only with values that keep it more general than S. Temperature=High gives <High,?,?>, Load=Heavy gives <?,Heavy,?>; the Vibration option <?,?,High> is rejected because it is not more general than S.
- **D4 (negative: High, Light, High):** <High,?,?> covers D4, so it is removed (its only specialisation <High,Heavy,?> is already below <?,Heavy,?>). G = {<?,Heavy,?>}.
- **D5 (positive):** S generalises to <?,Heavy,?>, so S = G and the version space has converged. **D6** changes nothing.
- **Result:** the learned concept is "Fails Early ⇔ Load = Heavy", identical from Find-S and Candidate-Elimination, and consistent with all six records.

1.3 Version space on D1–D3 and brute-force MAP (uniform prior)

The space H contains 3×3×3 = 27 conjunctions plus Ø, i.e. |H| = 28 hypotheses. Consistent with D1, D2 (positive) and D3 (negative): the hypothesis must cover D1 and D2, so Vibration = "?", Temperature ∈ {High, ?}, Load ∈ {Heavy, ?}, and it must exclude D3 (Normal, Light, Low), so at least one of Temperature = High or Load = Heavy must be kept. The version space therefore has exactly three members:

VS(D1–D3) = { h1 = <High,Heavy,?>, h2 = <High,?,?>, h3 = <?,Heavy,?> }

Noise-free likelihood: P(D|h) = 1 if h is consistent with every example, else 0. Uniform prior P(h) = 1/28. Evidence P(D) = Σ P(D|h)P(h) = 3/28.

Posterior with uniform prior

Hypothesis	P(h)	P(D|h)	P(h|D) = P(D|h)P(h)/P(D)
<High,Heavy,?>	1/28	1	1/3 = 0.3333
<High,?,?>	1/28	1	1/3 = 0.3333
<?,Heavy,?>	1/28	1	1/3 = 0.3333
other 25 hypotheses	1/28 each	0	0

1.4 Non-uniform prior, h_ML and h_MAP

Prior chosen (over the three version-space members): P(<High,?,?>) = 0.45, P(<?,Heavy,?>) = 0.35, P(<High,Heavy,?>) = 0.20. **Justification:** (i) a simplicity/Occam bias favours general hypotheses (fewer constraints = shorter description), so the two one-condition hypotheses get more mass than the two-condition one; (ii) between them, engineers' domain belief that temperature is the more common driver of early bearing failure gives Temperature a slight edge over Load.

Posterior with the non-uniform prior (D1–D3, noise-free)

Hypothesis	Prior P(h)	Likelihood P(D|h)	P(D|h)P(h)	Posterior P(h|D)
<High, Heavy, ?>	0.20	1	0.20	0.20
<High, ?, ?>	0.45	1	0.45	0.45
<?, Heavy, ?>	0.35	1	0.35	0.35

- **h_ML** = argmax P(D|h): the likelihood is 1 for all three, so h_ML is not unique (three-way tie).
- **h_MAP** = argmax P(D|h)P(h) = <High,?,?> (posterior 0.45). The prior breaks the tie.

Reasoning inquiry

When do h_ML and h_MAP coincide? When the prior is uniform (P(h) constant, so argmax P(D|h)P(h) = argmax P(D|h)), or more generally when the prior does not vary enough over the hypotheses to overturn the likelihood ordering; also when the likelihood already singles out one hypothesis with enough margin. With uniform prior and noise-free data, every consistent hypothesis ties; MAP and ML then coincide as the set of all maximisers.

A strong prior changing the selection (own calculation). Use all six records with a noisy likelihood P(D|h) = (1−ε)^(correct) ε^(errors), ε = 0.2. Errors on D1–D6: <?,Heavy,?> = 0, <High,Heavy,?> = 1 (D5), <High,?,?> = 2 (D4, D5). Likelihoods: 0.2621, 0.0655, 0.0164. Hence h_ML = <?,Heavy,?>.

Uniform prior: h_MAP = h_ML = <?,Heavy,?>

Hypothesis	Uniform prior	Likelihood	Posterior
<High, Heavy, ?>	0.333	0.0655	0.190
<High, ?, ?>	0.333	0.0164	0.048
<?, Heavy, ?>	0.333	0.2621	0.762

Strong prior (0.90 on <High,?,?>): h_MAP switches to <High,?,?>

Hypothesis	Strong prior	Likelihood	Posterior
<High, Heavy, ?>	0.05	0.0655	0.105
<High, ?, ?>	0.90	0.0164	0.474
<?, Heavy, ?>	0.05	0.2621	0.421

The flip occurs when the prior ratio exceeds the likelihood ratio, $(0.8/0.2)^2 = 16$: here 0.90/0.05 = 18 > 16, so posterior 0.474 (<High,?,?>) > 0.421 (<?,Heavy,?>) even though the data fit <High,?,?> worse. With more data the likelihood term grows exponentially and the prior is eventually overwhelmed.

MAP versus the whole version space. Take a new part x = (High, Light, Low). The three hypotheses vote: <High,Heavy,?> → No (0.20), <High,?,?> → Yes (0.45), <?,Heavy,?> → No (0.35). The single h_MAP = <High,?,?> classifies x as **Yes**, but the posterior-weighted vote is P(Yes) = 0.45 versus P(No) = 0.55, so the combined opinion (Bayes optimal) says **No**. A single hypothesis ignores the probability mass of the other hypotheses that disagree with it; h_MAP is optimal for choosing a hypothesis, not for classifying a new instance.

Q2 – MDL Principle, Bayes Optimal Classifier and Gibbs Algorithm

Part A – Model complexity (MDL)

log2(200) = 7.644 bits per misclassified record; 12 bits per node.

Description lengths (bits)

	L(h) = 12 × nodes	L(D|h) = errors × 7.644	Total L(h)+L(D|h)
Tree A (6 nodes, 8 errors)	72.00	61.15	133.15
Tree B (25 nodes, 1 error)	300.00	7.64	307.64

MDL selects Tree A (133.15 < 307.64 bits): Tree B saves only 7×7.644 = 53.5 bits of error description but pays 228 extra bits for its 19 additional nodes.

Node cost at which the preference reverses. With cost c bits per node: 6c + 61.15 = 25c + 7.64 ⇒ 19c = 53.51 ⇒ **c* = 2.82 bits**. Below 2.82 bits per node Tree B is preferred; above it Tree A.

Sensitivity of the choice to node cost

Bits per node c	L(Tree A)	L(Tree B)	Preferred
1	67.15	32.64	B
2	73.15	57.64	B
2.816 (c*)	78.05	78.04	tie
3	79.15	82.64	A
4	85.15	107.64	A
6	97.15	157.64	A
12	133.15	307.64	A
16	157.15	407.64	A

Link to MAP. h_MAP = argmax P(D|h)P(h) = argmin [−log2 P(D|h) − log2 P(h)]. Shannon coding says an event of probability p gets an optimal code of length −log2 p bits. Therefore L(h) = −log2 P(h) is the length of the optimal code for the hypothesis (the prior encodes the node cost: a prior that penalises large trees), and L(D|h) = −log2 P(D|h) is the optimal code for the data given h (for a classifier, the misclassified examples that must be transmitted as exceptions). Minimising the total description length is therefore the same as MAP selection.

Part B – Prediction under uncertainty

Hypothesis	P(h|D)	Prediction for X
h1	0.30	+
h2	0.25	−
h3	0.20	−
h4	0.15	+
h5	0.10	−

MAP versus Bayes optimal

Method	Weighted vote for +	Weighted vote for −	Classification of X
(i) MAP (h1 only)	1 (h1)	0	+
(ii) Bayes optimal	0.30 + 0.15 = 0.45	0.25 + 0.20 + 0.10 = 0.55	−

MAP and Bayes optimal disagree: h1 has the single largest posterior (0.30) but the three "−" hypotheses jointly hold 0.55.

Gibbs algorithm. (1) Draw one hypothesis h from the posterior P(h|D); (2) classify the new instance with that h alone. It costs one sample from the posterior and one evaluation instead of a sum over all of H.

Expected errors on X (true hypothesis drawn from the posterior; p = P(+) = 0.45, q = 0.55):

$$E[\text{error_Gibbs}] = P(\text{pred }+)P(\text{true }-) + P(\text{pred }-)P(\text{true }+) = 2pq = 2\times0.45\times0.55 = 0.495$$

$$E[\text{error_BO}] = \min(p, q) = 0.45$$

Relationship: E[error_BO] ≤ E[error_Gibbs] ≤ 2·E[error_BO]; check: 0.45 ≤ 0.495 ≤ 0.90 ✓. (In general 2pq ≤ 2·min(p,q) because max(p,q) ≤ 1.)

Simulation check (NumPy default_rng(42): independent draws of the predicting and the true hypothesis from the posterior)

Quantity	Analytical	Monte Carlo (N = 200,000, seed 42)
Gibbs expected error	0.4950	0.49401
Bayes optimal expected error	0.4500	0.45067

The simulation agrees with theory within sampling error (≈ ±0.0022 at 95 %).

Design inquiry

- **Bayes optimal may not equal any single hypothesis in H:** it is a posterior-weighted vote, a function that can lie outside H (e.g. here it outputs "−" although the most probable hypothesis says "+"; with conjunctions of literals a vote over hypotheses is generally not a conjunction).
- **Why rarely used directly:** it needs P(h|D) for every h and a sum over all of H at every prediction. Cost is O(|H|) per query and |H| is exponential (3^n+1 conjunctions; astronomically many trees or networks). It also needs the full posterior, which is itself intractable.
- **Practical advantage of Gibbs:** only one posterior sample and one hypothesis evaluation per prediction (e.g. via MCMC), with the guarantee that its expected error is at most twice that of the Bayes optimal classifier. It trades up to a factor of 2 in error for tractability.

Q3 – Naïve Bayes Classifier for Loan Approval

3.1 Assumption, decision rule and probability tables

Naïve Bayes assumption: features are conditionally independent given the class, $P(x1,x2,x3 \mid c) = \Pi_j P(x_j \mid c)$.
Decision rule: $\hat{y} = \text{argmax}_c P(c) \Pi_j P(x_j \mid c) = \text{argmax}_c [\log P(c) + \Sigma_j \log P(x_j \mid c)]$.

Class counts: Approve = Yes: 5 (IDs 1,2,3,7,8); No: 5. Priors **P(Yes) = P(No) = 0.5**. Conditional probability tables (maximum likelihood, no smoothing):

Income	P(· | Yes)	P(· | No)
High	2/5 = 0.40	1/5 = 0.20
Medium	2/5 = 0.40	1/5 = 0.20
Low	1/5 = 0.20	3/5 = 0.60

Employment	P(· | Yes)	P(· | No)
Salaried	4/5 = 0.80	1/5 = 0.20
Self-employed	1/5 = 0.20	4/5 = 0.80

Credit History	P(· | Yes)	P(· | No)
Good	4/5 = 0.80	1/5 = 0.20
Poor	1/5 = 0.20	4/5 = 0.80

3.2 Manual classification

(Low, Self-employed, Poor):

Yes: $0.5 \times (1/5) \times (1/5) \times (1/5) = 0.00400$ No: $0.5 \times (3/5) \times (4/5) \times (4/5) = 0.19200$

Normalised: $P(\text{Yes}) = 0.004/0.196 = 0.0204$, $P(\text{No}) = 0.9796$ → Reject

(Medium, Salaried, Good):

Yes: $0.5 \times (2/5) \times (4/5) \times (4/5) = 0.12800$ No: $0.5 \times (1/5) \times (1/5) \times (1/5) = 0.00400$

Normalised: $P(\text{Yes}) = 0.128/0.132 = 0.9697$, $P(\text{No}) = 0.0303$ → Approve

3.3 Unseen value Income = "Very High" and Laplace smoothing

Query (Very High, Salaried, Good). With maximum likelihood, P(Very High | Yes) = P(Very High | No) = 0/5 = 0, so both unnormalised scores are exactly 0 and the posterior is 0/0 (undefined): a single unseen value wipes out all other evidence (also in log-space, log 0 = −∞ for both classes).

Add-α smoothing: $P(v \mid c) = (\text{count} + \alpha)/(N_c + \alpha \cdot V)$, where V is the number of values of the feature (V = 4 for Income including "Very High"; V = 2 for Employment and Credit). **Chosen α = 1 (Laplace)**: the data set is tiny (5 examples per class), so a full pseudo-count is justified; it is the standard default and α can be tuned by cross-validation on larger data.

$P(\text{Very High} \mid \text{Yes}) = (0+1)/(5+4) = 0.1111$; $P(\text{Very High} \mid \text{No}) = 1/9 = 0.1111$; $P(\text{Salaried}|\text{Yes}) = 5/7$; $P(\text{Salaried}|\text{No}) = 2/7$; $P(\text{Good}|\text{Yes}) = 5/7$; $P(\text{Good}|\text{No}) = 2/7$

Yes: $0.5 \times (1/9)(5/7)(5/7) = 0.02834$ No: $0.5 \times (1/9)(2/7)(2/7) = 0.00454$ → $P(\text{Yes}) = 0.862$ → Approve

Effect of α (smaller α trusts the data more; larger α pulls toward 0.5)

α	P(Very High | class)	Unnormalised Yes	Unnormalised No	P(Yes | x)
0.5	0.0714	0.02009	0.00223	0.900

α	P(Very High | class)	Unnormalised Yes	Unnormalised No	P(Yes | x)
1	0.1111	0.02834	0.00454	0.862
2	0.1538	0.03419	0.00855	0.800

Smoothing also softens over-confident posteriors on the earlier queries: (Low, Self-employed, Poor) goes from P(No) = 0.980 to 0.926, and (Medium, Salaried, Good) from P(Yes) = 0.970 to 0.904; the decisions do not change.

3.4 From-scratch implementation and library verification

The class NB in nb.py (Appendix) estimates priors and CPTs with add-α smoothing and scores in log-space: score(c) = log P(c) + Σ log P(x_j|c), followed by a log-sum-exp style normalisation (subtract the max score before exponentiating). It was compared with scikit-learn CategoricalNB(alpha = 1) on the same data (categories fixed to include "Very High").

Verification (α = 1). Maximum absolute difference = 2.2×10^{-16}

Query	Mine P(Yes)	Library P(Yes)
(Low, Self-employed, Poor)	0.074074	0.074074
(Medium, Salaried, Good)	0.903614	0.903614
(Very High, Salaried, Good)	0.862069	0.862069

The hand calculations above (0.074, 0.904, 0.862 for P(Yes)) match both implementations.

3.5 Redundant-feature experiment

Data (seed 7, n = 2000, 70/30 split): binary class y ~ Bernoulli(0.5); four binary features generated conditionally on y with P(x=y) = 0.70 (x1), 0.60 (x2), 0.58 (x3), 0.55 (x4), independent given y. A fifth feature is (a) an exact duplicate of x1 or (b) a copy of x1 with 10 % of values flipped (90 % correlated). NB (α = 1) is trained with and without it.

Single run

Model (seed 7)	Test accuracy	Test log-loss
without redundant feature	0.700	0.5908
with exact duplicate of x1	0.700	0.6585
with 90%-correlated copy of x1	0.705	0.6356

Averaged over 30 data sets

Model (mean ± sd, 30 runs, seeds 100–129)	Accuracy	Log-loss
Without redundant feature	0.7018 ± 0.0155	0.5818 ± 0.0148
With exact duplicate of x1	0.6962 ± 0.0149	0.6445 ± 0.0254
With 90 %-correlated copy of x1	0.6935 ± 0.0185	0.6210 ± 0.0228

Validation inquiry

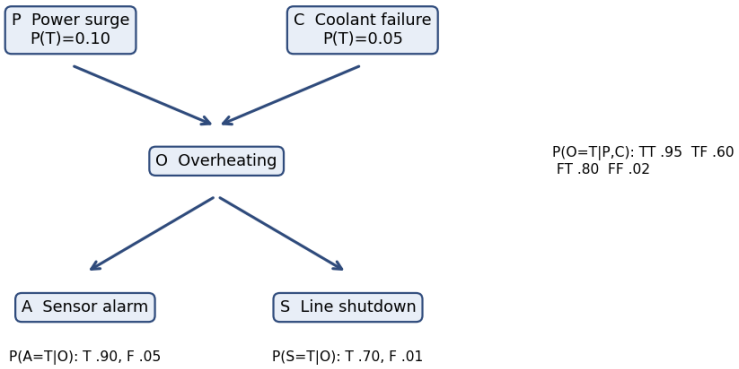
Does a violated independence assumption always reduce accuracy? No. Accuracy moved from 0.702 to 0.696 (exact duplicate) and 0.694 (correlated copy), a change smaller than one standard deviation (≈ 0.015). Accuracy depends only on which class has the larger score, and doubling the weight of one informative feature rarely flips that ordering. But the **probability estimates became markedly worse**: log-loss rose from 0.582 to 0.644 (+10.8 %) and 0.621. The duplicate counts the same evidence twice, so its log-odds contribution (ln(0.7/0.3) = 0.85 per observation) becomes 1.69 and the posterior is pushed toward 0 or 1: correct rankings, over-confident

probabilities. Hence a classifier can remain accurate yet be poorly calibrated; if the probabilities feed costs or thresholds, independence violations matter even when accuracy does not move.

Why log-space is necessary. The score is a product of many numbers below 1. With d features of probability ≈ 0.01, the product is 10^(−2d) and underflows IEEE double precision (smallest ≈ 10^−308) at d ≈ 150, giving 0 for every class so that the argmax and the normalisation are lost. Summing logs is numerically stable, turns the rule into an addition (a linear model), and normalisation uses the log-sum-exp trick (subtract the largest log-score before exponentiating).

Q4 – Bayesian Belief Network for the Production Line

4.1 Network, factorisation and parameter count



Network: P → O ← C, O → A, O → S

$$P(P,C,O,A,S) = P(P) \cdot P(C) \cdot P(O \mid P,C) \cdot P(A \mid O) \cdot P(S \mid O)$$

Independent parameters: P: 1; C: 1; O|P,C: 4 (one per parent configuration); A|O: 2; S|O: 2. **Total = 10.** The full joint over five binary variables needs $2^5 - 1 =$ **31** parameters. The network cuts the parameter count to about one third, and the saving grows exponentially with the number of variables for sparse graphs.

4.2 d-separation

Conditional independence claims

Statement	Holds?	d-separation reasoning	Numerical check (max $\lvert P(x,y\mid z) - P(x\mid z)P(y\mid z)\rvert$)
A ⊥ S | O	Yes	Only path A ← O → S passes through O, a common cause (fork). Observing O blocks it.	1.1e-16
P ⊥ C | (nothing)	Yes	Only path P → O ← C has O as a collider. A collider blocks unless it or a descendant (A, S) is observed; nothing is observed.	0.0e+00
P ⊥ C | O	No	Conditioning on the collider O opens the path P → O ← C.	0.149
P ⊥ C | A	No	A is a descendant of the collider O; observing a descendant also opens the path.	0.069

4.3 Manual inference

$P(O = T) = \Sigma_{P,C} P(P)P(C)P(O=T|P,C)$

P	C	P(P)P(C)	P(O=T | P,C)	Product
T	T	0.0050	0.95	0.00475
T	F	0.0950	0.60	0.05700
F	T	0.0450	0.80	0.03600
F	F	0.8550	0.02	0.01710

$$P(O = T) = 0.00475 + 0.05700 + 0.03600 + 0.01710 = 0.11485$$

P(C = T | A = T). Sum out P and O (S sums to 1 and drops out). First P(O=T | C) for each C:

$$P(O=T \mid C=T) = 0.1\times0.95 + 0.9\times0.80 = 0.815 \; ; \quad P(O=T \mid C=F) = 0.1\times0.60 + 0.9\times0.02 = 0.078$$

$$P(A=T \mid C=T) = 0.815\times0.90 + 0.185\times0.05 = 0.74275 \; ; \quad P(A=T \mid C=F) = 0.078\times0.90 + 0.922\times0.05 = 0.1163$$

$$P(A=T, C=T) = 0.05\times0.74275 = 0.0371375 \;;\quad P(A=T, C=F) = 0.95\times0.1163 = 0.110485$$

$$P(C=T \mid A=T) = 0.0371375 / (0.0371375 + 0.110485) = 0.0371375 / 0.1476225 = 0.2516$$

4.4 Explaining away

With P = T known: $P(A=T, C=T, P=T) = 0.1\times0.05\times(0.95\times0.90 + 0.05\times0.05) = 0.0042875$ and $P(A=T, C=F, P=T) = 0.1\times0.95\times(0.60\times0.90 + 0.40\times0.05) = 0.0532$.

$$P(C=T \mid A=T, P=T) = 0.0042875 / 0.0574875 = 0.0746$$

Interpretation. The alarm raises the belief in coolant failure from the prior 0.05 to 0.2516 because overheating (which triggers the alarm) needs a cause. Once we also learn that a power surge occurred, the surge already accounts for the overheating, so the alarm gives much less reason to suspect the coolant: P(C | A, P) = 0.0746, close to the prior 0.05. P and C, independent a priori, become competing explanations of the same effect once the effect (or its descendant A) is observed; confirming one explanation lowers the probability of the other. This is the "explaining away" (Berkson-type) effect of a collider and cannot be represented by Naïve Bayes.

4.5 Inference by enumeration (code) and verification

All manual answers are reproduced; the full joint sums to 1.000000

Query	Manual	Coded	Joint terms summed
P(O=T)	0.11485	0.11485	32 enumerated (16 with O = T in the numerator)
P(C=T | A=T)	0.2516	0.2516	16 (A fixed; sum over P,C,O,S)
P(C=T | A=T, P=T)	0.0746	0.0746	8 (A,P fixed; sum over C,O,S)
P(P=T | A=T) (extra)	–	0.3894	16

4.6 Simulating the network and learning CPTs by maximum likelihood

1,000 records were sampled with NumPy default_rng(42) (ancestral sampling P, C → O → A, S). CPTs were estimated by relative frequency from the first 50, 200 and 1,000 records. Estimates of P(O = T | P, C), with the number k of records in each parent cell:

Estimates of P(O=T | P,C)

Parent cell	True	n = 50	n = 200	n = 1000
P=T, C=T	0.95	n/a (k=0)	1.000 (k=1, err 0.050)	1.000 (k=7, err 0.050)
P=T, C=F	0.60	1.000 (k=3, err 0.400)	0.882 (k=17, err 0.282)	0.602 (k=88, err 0.002)
P=F, C=T	0.80	0.500 (k=2, err 0.300)	0.800 (k=5, err 0.000)	0.903 (k=31, err 0.103)
P=F, C=F	0.02	0.022 (k=45, err 0.002)	0.011 (k=177, err 0.009)	0.016 (k=874, err 0.004)

Averaged error over 200 independent samples of each size (seeds 1000–1199)

Sample size	Mean abs. error of P(O|P,C) – 200 replicate samples	Share of parent cells with no data
50	0.1380	20.9 %
200	0.0744	8.5 %
1000	0.0440	0.1 %

Error falls with sample size (≈ $1/\sqrt{k}$), but the rare parent configurations are the problem: P(P=T, C=T) = 0.005 means k = 0 at n = 50 (estimate undefined) and k = 7 at n = 1000, while the dominant cell (F,F) is estimated well at all sizes. Maximum-likelihood CPTs therefore need smoothing/Dirichlet priors for rare parent states.

Design inquiry

Comparison with Naïve Bayes (Q3). NB is a belief network with a fixed star structure (class → every feature): it assumes the features are conditionally independent given the class. A belief network **relaxes this assumption**: any DAG is allowed, so variables may be dependent given a parent set, chain effects (O → A and O → S), and collider structure (P, C independent a priori but dependent given O or A) can be modelled, giving explaining away that NB cannot express. **Cost of exact inference:** enumeration costs $O(2^n)$ joint terms (32 here, 2^n for n variables); variable elimination is exponential in the treewidth, and exact inference is NP-/#P-hard for general graphs. CPT size also grows exponentially with the number of parents. For large networks one uses approximate inference (sampling, loopy belief propagation) or restricts the structure (trees, TAN).

Q5 – EM Algorithm for a Gaussian Mixture of Two Machines

5.1 Model, hidden variable, E-step and M-step

Mixture: $p(x) = \Sigma_{k=1,2} \pi_k N(x \mid \mu_k, \sigma_k^2)$, with $\pi_1 + \pi_2 = 1$. **Hidden variable** $z_i \in \{1,2\}$: the (unrecorded) machine that produced reading x_i, with $P(z_i = k) = \pi_k$.

E-step (responsibilities):

$$r_{ik} = P(z_i = k \mid x_i, \theta) = \pi_k N(x_i \mid \mu_k, \sigma_k^2) / \Sigma_j \pi_j N(x_i \mid \mu_j, \sigma_j^2)$$

M-step, with $N_k = \Sigma_i r_{ik}$:

$$\pi_k = N_k / N \; ; \quad \mu_k = (1/N_k) \Sigma_i r_{ik} x_i \; ; \quad \sigma_k^2 = (1/N_k) \Sigma_i r_{ik} (x_i - \mu_k)^2$$

Derivation of the μ_k update. The expected complete-data log-likelihood is

$$Q(\theta \mid \theta_{old}) = \Sigma_i \Sigma_k r_{ik} [\ln \pi_k - \tfrac{1}{2} \ln(2\pi \sigma_k^2) - (x_i - \mu_k)^2 / (2\sigma_k^2)]$$

Differentiating with respect to μ_k (only the last term depends on it) and setting to zero:

$$\partial Q/\partial \mu_k = \Sigma_i r_{ik} (x_i - \mu_k)/\sigma_k^2 = 0 \Rightarrow \Sigma_i r_{ik} x_i = \mu_k \Sigma_i r_{ik} \Rightarrow \mu_k = \Sigma_i r_{ik} x_i / \Sigma_i r_{ik}$$

(The same procedure on σ_k^2 gives the weighted variance; for π_k maximise $\Sigma_i \Sigma_k r_{ik} \ln \pi_k$ subject to $\Sigma \pi_k = 1$ with a Lagrange multiplier, giving $\pi_k = N_k/N$.) The new mean is a responsibility-weighted average of the readings.

5.2 One manual EM iteration on the seven readings

Start: $\pi1 = \pi2 = 0.5$, $\mu1 = 2$, $\mu2 = 8$, $\sigma1^2 = \sigma2^2 = 1$. $N(x|\mu,1) = \exp(-(x-\mu)^2/2)/\sqrt{(2\pi)}$.

E-step: unnormalised terms and responsibilities (x = 5.0 is equidistant, r = 0.5/0.5)

x	π1·N(x|2,1)	π2·N(x|8,1)	r1 (machine 1)	r2 (machine 2)
1.8	0.195521	8.97e-10	1.000000	0.000000
2.5	0.176033	5.38e-08	1.000000	0.000000
3.1	0.108926	1.22e-06	0.999989	0.000011
5.0	0.002216	2.22e-03	0.500000	0.500000
7.6	0.000000	1.84e-01	0.000000	1.000000
8.4	0.000000	1.84e-01	0.000000	1.000000
9.0	0.000000	1.21e-01	0.000000	1.000000

M-step: N1 = 3.5000, N2 = 3.5000.

$$\pi1 = 0.5000, \quad \pi2 = 0.5000$$

$$\mu1 = (1.8 + 2.5 + 3.1 + 0.5\times5.0 + \ldots)/3.5 = 2.8286, \quad \mu2 = 7.8571$$

$$\sigma1^2 = 1.0278, \quad \sigma2^2 = 1.6425$$

Log-likelihood before = -16.5014 → after = -14.9842 (increase of 1.5172)

5.3 Implementation, monotonic likelihood and convergence rule

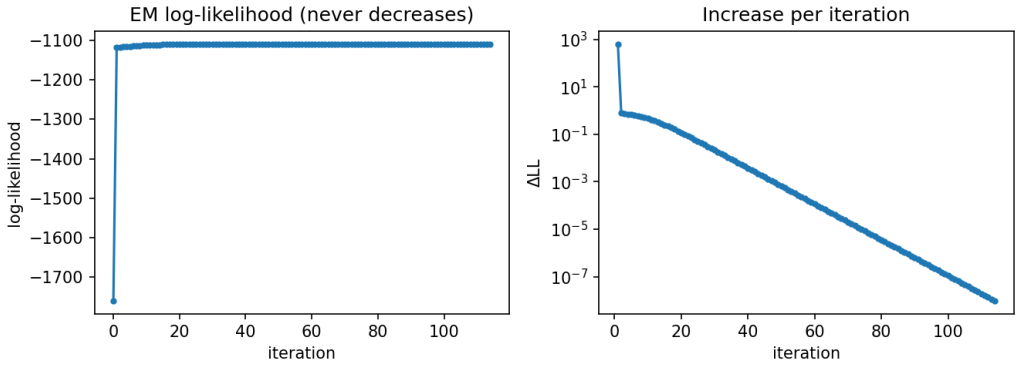
Data: 500 observations (NumPy default_rng(2024)) from a known mixture $\pi = (0.4, 0.6)$, $\mu = (2, 6)$, $\sigma = (1, 1.5)$ ($\sigma^2 = 1, 2.25$). **Convergence rule:** stop when $|LL_t - LL_{t-1}| < 10^{-8}$ (maximum 1000 iterations); variances floored at 10^{-6} to avoid singularities.

Converged in 114 iterations, final LL = -1108.478

	π	μ	σ²
True	(0.4, 0.6)	(2.00, 6.00)	(1.000, 2.250)

	π	μ	σ²
EM estimate	(0.366, 0.634)	(2.001, 6.022)	(0.956, 2.610)

The smallest per-iteration change in log-likelihood over the whole run was 9.60e-09 ≥ 0, so the log-likelihood **never decreased**.



Log-likelihood is non-decreasing at every iteration

5.4 Ten random initialisations (plus two deliberately poor starts)

Random inits: means ~ U(min,max), variances ~ U(0.5,5), π random; seeds 0–9

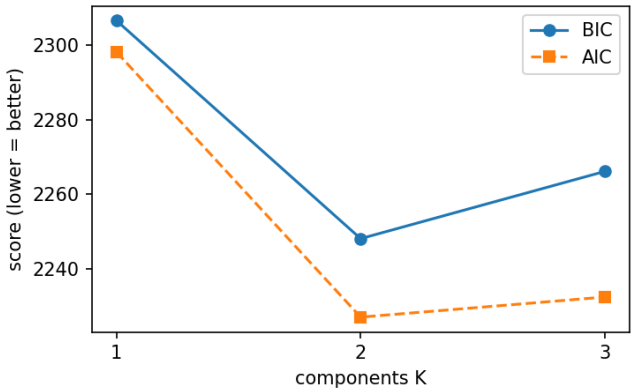
Run	Initial μ	Final μ	Final σ²	Final π	Final LL	Iter.	Labels swapped?
rand0	(6.31, 2.30)	(6.022, 2.001)	(2.610, 0.956)	(0.634, 0.366)	-1108.478	102	yes
rand1	(4.94, 9.74)	(2.001, 6.022)	(0.956, 2.610)	(0.366, 0.634)	-1108.478	135	no
rand2	(2.21, 2.61)	(6.022, 2.001)	(2.610, 0.956)	(0.634, 0.366)	-1108.478	110	yes
rand3	(0.28, 1.93)	(2.001, 6.022)	(0.956, 2.610)	(0.366, 0.634)	-1108.478	130	no
rand4	(9.66, 4.94)	(6.022, 2.001)	(2.610, 0.956)	(0.634, 0.366)	-1108.478	143	yes
rand5	(8.15, 8.18)	(2.001, 6.022)	(0.956, 2.610)	(0.366, 0.634)	-1108.478	122	no
rand6	(5.23, 3.10)	(6.022, 2.001)	(2.610, 0.956)	(0.634, 0.366)	-1108.478	98	yes
rand7	(6.18, 9.16)	(2.001, 6.022)	(0.956, 2.610)	(0.366, 0.634)	-1108.478	150	no
rand8	(2.92, 10.15)	(2.001, 6.022)	(0.956, 2.610)	(0.366, 0.634)	-1108.478	136	no
rand9	(8.87, 2.48)	(6.022, 2.001)	(2.610, 0.956)	(0.634, 0.366)	-1108.478	123	yes
poor-A (both means at 4.0)	(3.90, 4.10)	(2.001, 6.022)	(0.956, 2.610)	(0.500, 0.500)	-1108.478	139	no
poor-B (means 10, 11)	(10.00, 11.00)	(4.379, 8.024)	(5.410, 0.105)	(0.500, 0.500)	-1141.613	63	no

- All 10 random starts reached the same maximum (LL = −1108.478), differing only in the 8th decimal and in 98–150 iterations.
- **Label switching:** in 5 of the 10 runs (rand0, 2, 4, 6, 9) component 1 ended up with the larger mean (6.02) and component 2 with the smaller (2.00): the same solution with labels permuted (the likelihood is symmetric under swapping component indices), not a different optimum.
- **Local optimum:** the poor start "means 10 and 11" converged to LL = −1141.61, 33.1 log-likelihood units below the global value, with μ = (4.38, 8.02) and a very narrow component (σ² = 0.10) fitted to a tight cluster of readings: a genuine local maximum. The start with both means at ≈ 4 still reached the global optimum.

5.5 Model selection: 1, 2 and 3 components

Best of 10 restarts per K (n = 500, p = 3K − 1)

K	Free parameters	Log-likelihood	BIC = −2LL + p ln n	AIC = −2LL + 2p
1	2	-1147.06	2306.54	2298.11
2	5	-1108.48	2248.03	2226.96
3	8	-1108.17	2266.07	2232.35

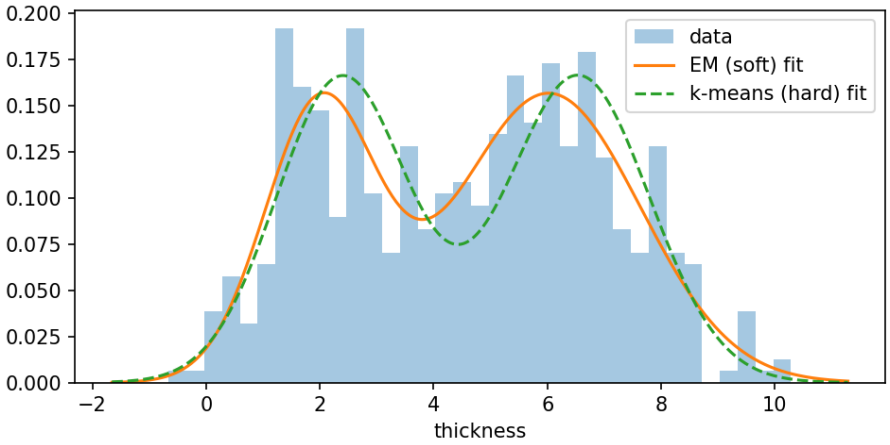


Going from K = 1 to 2 raises LL by 38.6, easily paying for 3 extra parameters (BIC 2306.5 → 2248.0). Going from K = 2 to 3 raises LL by only 0.30 while adding 3 parameters, so BIC rises to 2266.1 (AIC also worsens). **The data support K = 2**, matching the generating process.

5.6 Soft (EM) versus hard (k-means) assignment

Same data; the k-means parameters are the cluster means, proportions and variances

	μ	π	σ^2	Log-likelihood	Accuracy vs true machine
Truth	(2.00, 6.00)	(0.40, 0.60)	(1.00, 2.25)	–	–
EM (soft)	(2.00, 6.02)	(0.37, 0.63)	(0.96, 2.61)	-1108.48	94.4 %
k-means (hard)	(2.39, 6.54)	(0.48, 0.52)	(1.34, 1.56)	-1116.72	88.8 %



Hard assignment places the boundary midway between the means regardless of component weights and widths, assigns the overlapping readings entirely to one machine, and therefore biases the means toward each other's side (2.39 and 6.54 versus the true 2 and 6), underestimates the larger spread, and misclassifies more points (88.8 % vs 94.4 %). EM weights each reading by its responsibility, uses the unequal variances and priors, and attains a higher likelihood (−1108.5 versus −1116.7).

Analysis inquiry

- **Why no decrease.** For any distribution q over z, $\ln p(X|\theta) = L(q,\theta) + KL(q \,\|\, p(Z|X,\theta))$, where $L(q,\theta) = \Sigma\, q \ln[p(X,Z|\theta)/q]$ is a lower bound. The E-step sets $q = p(Z|X,\theta_old)$, making KL = 0 so $L(q,\theta_old) = \ln p(X|\theta_old)$. The M-step maximises L over θ, so $L(q,\theta_new) \geq L(q,\theta_old)$. Then $\ln p(X|\theta_new) \geq L(q,\theta_new) \geq L(q,\theta_old) = \ln p(X|\theta_old)$.
- **Why not a global maximum.** Each step only guarantees improvement from the current point; the mixture likelihood is non-convex and EM converges to a stationary point (local maximum or saddle) that depends on the initialisation, as the "means 10 and 11" run shows (−1141.6 versus −1108.5). Remedies: multiple restarts, k-means initialisation, variance floors.
- **Relation to MLE.** EM is an algorithm for computing the maximum-likelihood estimate (or MAP, if a prior is added) when the likelihood involves latent variables and has no closed form. If z were observed, the M-step is exactly the closed-form MLE (sample proportions, means and variances per component); EM replaces the missing counts by expected counts.
- **Learning the CPTs in the Q4 network with an unobserved variable** (say O hidden). E-step: for each record, compute the posterior P(O | P, C, A, S) by inference (enumeration). Accumulate expected counts $n(O=o, P=p, C=c) = \Sigma_records\ P(o \mid record)\cdot 1[P=p, C=c]$, and likewise for A and S given O. M-step: $P(O=T \mid p,c) = n(O=T,p,c)/n(p,c)$, $P(A=T \mid o) = n(A=T,o)/n(o)$, etc. Iterating increases the likelihood of the observed data exactly as for the mixture.

Q6 – Sample Complexity, Hypothesis-Space Capacity and Mistake Bounds

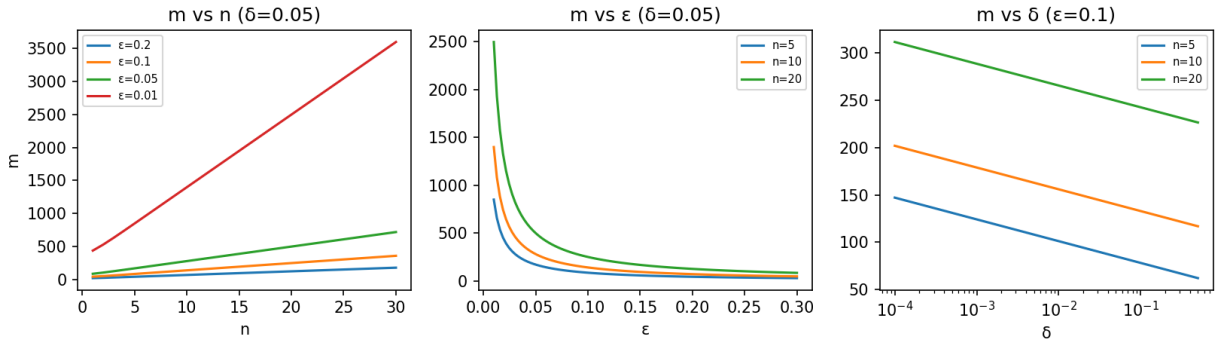
Part A – PAC learning and finite hypothesis spaces

PAC framework. A concept class C is PAC-learnable by learner L (using hypothesis space H) if, for every target $c \in C$, every distribution D over instances, and every $\varepsilon, \delta \in (0, \tfrac{1}{2})$, L outputs, with probability at least $1 - \delta$, a hypothesis h with $\text{error}_D(h) = P_D[h(x) \neq c(x)] \leq \varepsilon$, using time and number of examples polynomial in $1/\varepsilon$, $1/\delta$, n and size(c). ε is the accuracy parameter; δ is the confidence (failure probability) parameter.

For consistent learners over a finite H: $m \geq (1/\varepsilon)(\ln|H| + \ln(1/\delta))$. Conjunctions over n = 10 variables: $|H| = 3^{10} + 1 = 59{,}050$, $\ln|H| = 10.986$.

Sample sizes, n = 10

ε	δ	m (exact)	m (rounded up)
0.1	0.05	139.82	140
0.1	0.01	155.91	156
0.05	0.05	279.64	280
0.05	0.01	311.83	312



Bound versus n (left), ε (middle) and δ (right)

Strongest influence. The bound is linear in $1/\varepsilon$ and in n (through $\ln|H| = n \ln 3 + \ldots$), but only logarithmic in $1/\delta$. Halving ε doubles m (×2.00); doubling n multiplies it by 1.79; dividing δ by 10 increases it only ×1.16. **ε has the strongest influence** (with n second); δ is cheap.

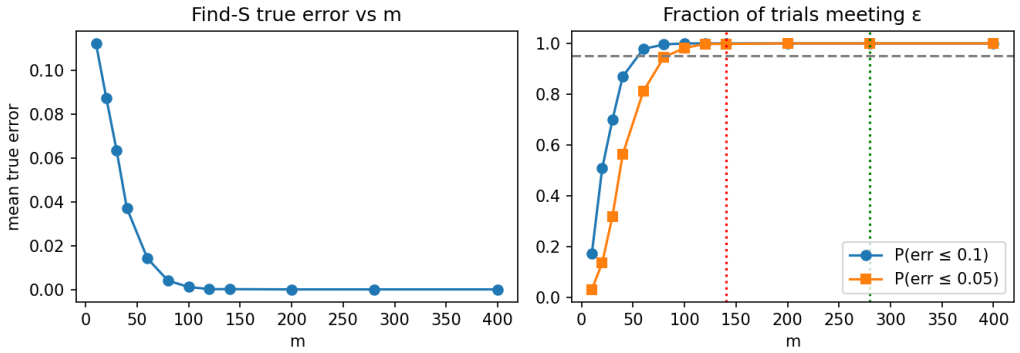
Find-S simulation

Target conjunction $c = x1 \wedge \neg x3 \wedge x5$ over n = 10 Boolean features ($P(c = 1) = 0.125$); examples uniform on $\{0,1\}^{10}$; noise-free labels. Find-S starts with all 20 literals and deletes any literal violated by a positive example. True error is computed exactly: $\text{error} = P(c) - 2^{(-|h|)}$, where $|h|$ is the number of literals left. 500 trials per m, seed 11.

Find-S results

m	Mean true error	P(error ≤ 0.10)	P(error ≤ 0.05)
10	0.1124	0.174	0.032
20	0.0876	0.508	0.136
30	0.0637	0.700	0.320
40	0.0372	0.870	0.564
60	0.0141	0.978	0.812
80	0.0040	0.996	0.946
100	0.0011	1.000	0.982
120	0.0001	1.000	0.998

m	Mean true error	P(error ≤ 0.10)	P(error ≤ 0.05)
140	0.0001	1.000	0.998
200	0.0000	1.000	1.000
280	0.0000	1.000	1.000
400	0.0000	1.000	1.000



Red dotted line: bound for ε = 0.1 (m = 140); green: ε = 0.05 (m = 280)

Check of the guarantee. At the bound for ε = 0.10, δ = 0.05 (m = 140) the fraction of trials with error ≤ 0.10 is 1.000 ≥ 0.95 ✓; at the bound for ε = 0.05, δ = 0.05 (m = 280) the fraction with error ≤ 0.05 is 1.000 ≥ 0.95 ✓. **The bound is conservative**: 95 % success is already reached at m ≈ 60 for ε = 0.10 (0.978; 0.87 at m = 40) and m ≈ 100 for ε = 0.05 (0.982; 0.946 at m = 80), so the bound is about 2.3–2.8 times larger than necessary here. The reasons: it holds for every distribution and every target, uses a union bound over all "bad" hypotheses in H, and ln|H| counts hypotheses irrespective of the structure Find-S actually exploits (uniform distribution, only 3 relevant literals).

Part B – Infinite hypothesis spaces

Shattering. A set S of points is shattered by H if for every one of the 2^|S| labellings of S some h ∈ H realises it. **VC(H)** is the size of the largest finite set shattered by H (∞ if arbitrarily large sets are shattered).

VC dimension arguments

Class	Shattered set (explicit)	Why no larger set is shattered	VC	Code check
Intervals [a,b] on the real line	{1, 2}: labels (−,−) interval away from both; (+,−) [0.5,1.5]; (−,+) [1.5,2.5]; (+,+) [1,2]	For x1<x2<x3 the labelling (+,−,+) would need an interval containing x1 and x3 but not x2: impossible	2	2 pts shattered: True; 3 pts: False
Linear separators in the plane	(0,0), (1,0), (0,1) (non-collinear): all 8 labellings by a line	Four points: in convex position the "XOR" labelling (diagonal pair +) is not separable; if one point lies inside the triangle of the others, labelling the inner point − and the three vertices + fails (Radon)	3	3 pts: True; XOR 4 pts: False
Axis-aligned rectangles in the plane	Diamond: (0,1), (0,−1), (1,0), (−1,0): the bounding box of any subset excludes the others	For any 5 points, label the (≤ 4) points with extreme x or y as + and a remaining point −: any rectangle containing the extremes contains their bounding box, hence that point	4	4 pts: True; 5 pts: False

VC-based sample bound m ≥ (1/ε)(4 log2(2/δ) + 8·VC·log2(13/ε)), ε = 0.1, δ = 0.05: 4 log2(40) = 21.29, log2(130) = 7.022.

VC sample bounds

Class	VC	m (exact)	m (rounded up)
Intervals on R	2	1336.5	1337
Linear separators in R^2	3	1898.2	1899
Axis-aligned rectangles in R^2	4	2460.0	2461

Comparison with Part A. The finite bound depends on ln|H| (here 11.0 → m = 140 for ε = 0.1, δ = 0.05) and is unavailable for infinite classes such as intervals, lines or rectangles, where |H| = ∞. The VC bound replaces ln|H| by the combinatorial capacity VC(H), so it applies to infinite H and grows linearly with VC and with log(1/ε) rather than ln|H|; it is also much looser in constants (8·VC·log2(13/ε)): for d = 2 it asks for ≈ 1,337 examples versus ≈ 140 for the (richer-looking) finite class of conjunctions. It is a worst-case, distribution-free guarantee that is usually pessimistic in practice.

Part C – Mistake bound model

Halving algorithm. H = all 64 monotone conjunctions over six Boolean variables (every subset of variables). The algorithm predicts with the majority vote of the current version space and discards inconsistent hypotheses after every example. Each mistake removes at least half of the version space, so mistakes ≤ log2|H| = 6.

Uniform random examples (60 per run, random target, seed 5)

Mistakes	0	1	2	3	4	5
Number of runs (of 2000)	134	469	860	465	71	1

Observed: maximum 5 mistakes, mean 1.94, against the bound log2 64 = 6. With biased examples (each bit = 1 with probability 0.8) the maximum was 6 (the bound is attained) and the mean 2.58. The bound is never violated.

Online conjunction learner. Start with the most specific hypothesis x1 ∧ ¬x1 ∧ … ∧ xn ∧ ¬xn (2n literals), which predicts negative for everything. On a positive example, delete every literal that the example violates. **Why ≤ n + 1 mistakes:** (1) the hypothesis always contains all literals of the target, so it never predicts positive when the target is negative: all mistakes are false negatives on positive examples; (2) the first mistake removes exactly n literals (one of x_i, ¬x_i for each i), leaving n; (3) every later mistake removes at least one literal and at most n literals can be removed before the target's literals are all that remain, so at most n further mistakes: total ≤ n + 1.

Empirical check (200 examples per sequence, half forced positive, seed 9)

n	Bound n+1	Max mistakes over 5000 random sequences	Mean mistakes	Adversarial sequence
5	6	6	2.68	6 (tight)
10	11	8	3.47	11 (tight)
20	21	9	4.35	21 (tight)

The bound is never exceeded and is met exactly by the adversarial sequence (target = always-true, first example all ones, then examples that each flip one more bit), which shows it is tight.

Relation VC(C) ≤ Opt(C) ≤ log2|C|. Opt(C) is the optimal worst-case mistake bound over all online learners. VC ≤ Opt: an adversary can present the d points of a shattered set one by one and, after each prediction, choose the opposite label (some concept in C realises it), forcing d mistakes. Opt ≤ log2|C|: Halving achieves it. Verified by exhaustive minimax computation on small concept classes:

Checks (Opt by recursive minimax)

Concept class	|C|	VC(C)	Opt(C)	log2|C|	VC ≤ Opt ≤ log2|C|?
Thresholds on 4 ordered points (5 concepts)	5	1	2	2.32	✓
All subsets of 3 points (8 concepts)	8	3	3	3.00	✓

Concept class	|C|	VC(C)	Opt(C)	log2|C|	VC ≤ Opt ≤ log2|C|?
Singletons + empty on 3 points (4 concepts)	4	1	1	2.00	✓

The threshold class has VC = 1 < Opt = 2 < log2 5 = 2.32 (strict gaps); all subsets of three points gives equality 3 = 3 = 3.

Selection inquiry

PAC (number of training examples): a batch guarantee: after m i.i.d. examples from a fixed distribution the hypothesis has error ≤ ε with probability ≥ 1 − δ. **Mistake bound (number of mistakes):** an online, worst-case guarantee on the total number of wrong predictions while learning, with no distributional assumption (examples may even be adversarial). **For the quality team:** if they can collect a labelled batch of inspection records from a stable process before deployment, the PAC bound tells them how many records to collect (for n = 10, ε = 0.1, δ = 0.05: ≈ 140; the simulation shows ≈ 60 suffices in practice). If the learner must operate on the live line, with inspection results arriving one at a time and each missed fault being costly, the mistake bound (≤ n + 1 = 11 mistakes, independent of the order of parts) is the more useful statement, and online-to-batch conversion turns it into a PAC guarantee as well. Choose PAC for stationary batch data with an acceptable error rate; choose mistake bounds when the data stream is not i.i.d. or total errors during learning matter.

Assumptions of my simulations: noise-free labels; a realisable, deterministic target (a conjunction, in H); independent, uniformly distributed Boolean features and examples drawn i.i.d. from that distribution (fixed during training and testing); Halving tested on a finite H containing the target; Find-S and the online learner do not tolerate label noise (one wrong positive label can delete needed literals). Results would change with noisy labels, a non-realisable concept or a shifting process.

Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate

Selected solution: the Naïve Bayes loan/synthetic classifier of Q3 (unreliable probability estimates caused by a violated independence assumption).

A. Diagnose the bottleneck

- **Evidence (Q3, 30 runs):** removing the redundant feature gives log-loss 0.5818 ± 0.0148; adding an exact duplicate of x1 raises it to 0.6445 ± 0.0254 (+10.8 %), a 90 %-correlated copy to 0.6210. Accuracy stays within noise (0.702 → 0.696). Expected calibration error (10 bins) of the duplicate model is 0.139 versus 0.039 after removal.
- **Cause: model assumption**, not data size or prior: n_train = 1400 for 11 parameters, α = 1 smoothing has negligible effect, and the CPTs themselves are accurate. The conditional dependence between x1 and its copy ($I(x1 ; x1' \mid Y) \approx 0.60$ nats for an exact duplicate, versus ≤ 0.0014 nats for the genuinely independent pairs) makes NB add the same log-likelihood ratio, ln(0.7/0.3) = 0.85, twice (1.69), pushing posteriors toward 0/1: right ranking, over-confident probabilities (reliability diagram below).

B. Design objective (set before redesign)

Measure	Baseline (exact duplicate, mean of 30 runs)	Target	Allowed trade-off
Test log-loss	0.6445	≤ 0.600 (recover ≥ 70 % of the 0.063 gap to the clean value 0.582)	–
Expected calibration error	0.139	≤ 0.060	–
Accuracy	0.6962	not lower than baseline by more than 1 percentage point	up to 1 pp
Parameters	11	≤ 11	extra $O(d^2 \cdot N)$ preprocessing allowed

C. Revised learning strategy: CMI-pruned Naïve Bayes

What changes and why. The diagnosis is conditional dependence between features given the class, so the redesign detects and removes redundant features before fitting rather than replacing NB with a more complex model. Redundancy is measured with conditional mutual information, $I(X_i ; X_j \mid Y) = \Sigma_y P(y)\, I(X_i ; X_j \mid y)$, estimated on the training set only. If it exceeds a threshold τ = 0.05 nats (≈ 100× the noise level 1/(2N) ≈ 0.0004 for N = 1400), the feature with lower $I(X ; Y)$ is discarded. NB is then fitted on the remaining features, so the independence assumption holds approximately. Addresses: violated independence. Needs: $O(d^2)$ pairwise statistics (memory $O(d^2)$), one threshold hyper-parameter. New limits: information in partially redundant features is discarded; τ must be chosen; only pairwise dependence is detected.

```
Pruned-NB(train X, y; τ = 0.05)
  keep ← all features
  rel[j] ← I(X_j ; Y)  for every j
  repeat
    (i, j) ← pair in keep with the largest I(X_i ; X_j | Y)
    if I(X_i ; X_j | Y) > τ:   remove from keep the one with lower rel[]
    else stop
  fit Naive Bayes (α = 1) on features in keep
  predict with log-space posterior
Workflow:  data → CMI matrix → drop redundant features → NB → probabilities
```

Alternative baseline included for comparison: NB + temperature scaling (probability recalibration): $p' = \sigma(\text{logit}(p)/T)$, with T fitted on the training log-loss. It does not change the ranking (so accuracy cannot change) and does not reduce parameters.

D. Prediction recorded before running the validation

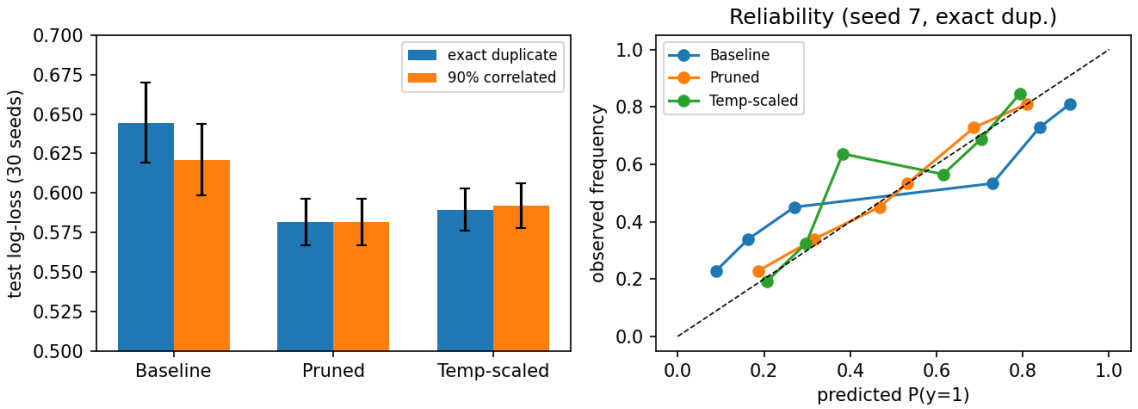
- Pruned NB will delete the duplicate (and the correlated copy), so its log-loss should return to ≈ 0.58 (within ±0.01 of the clean model), ECE should fall below 0.05, accuracy should stay within ±0.5 pp, and parameters drop 11 → 9.
- Temperature scaling should improve log-loss to about 0.59–0.60 and ECE below 0.06 but stay slightly behind pruning because one global T cannot undo double counting of a single feature; accuracy exactly unchanged.

E. Implementation and validation

Same data generator, seeds (7 for the single run; 100–129 for averages), 70/30 splits and metrics as in Q3. Averages over 30 data sets (mean ± sd):

Comparison on identical data sets

Scenario	Model	Accuracy	Log-loss	ECE	Params	Better log-loss than baseline
Exact duplicate	Baseline NB	0.6962 ± 0.0149	0.6445 ± 0.0254	0.139	11	0/30
Exact duplicate	CMI-pruned NB	0.7018 ± 0.0155	0.5818 ± 0.0148	0.039	9	30/30
Exact duplicate	NB + temp. scaling	0.6962 ± 0.0149	0.5894 ± 0.0135	0.052	12	30/30
90 % correlated	Baseline NB	0.6935 ± 0.0185	0.6210 ± 0.0228	0.106	11	0/30
90 % correlated	CMI-pruned NB	0.7018 ± 0.0155	0.5818 ± 0.0148	0.039	9	30/30
90 % correlated	NB + temp. scaling	0.6935 ± 0.0185	0.5921 ± 0.0143	0.045	12	30/30



Left: test log-loss (30 runs). Right: reliability diagram, seed 7, exact duplicate

Single run (seed 7, exact duplicate): baseline log-loss 0.6585, ECE 0.143; pruned 0.5908, ECE 0.044 (features kept: x1–x4); temperature scaling 0.5934 (T = 1.85). Fit time was a few milliseconds for every model. **False-positive check:** on the clean four-feature data the pruner kept 4 of 4 features in all 30 runs.

F. Justification of the final design

Objective	Target	Achieved (exact duplicate)	Met?
Log-loss	≤ 0.600	0.5818	Yes
ECE	≤ 0.060	0.039	Yes
Accuracy change	≥ −1 pp	+0.57 pp	Yes
Parameters	≤ 11	9	Yes

- **Prediction versus outcome:** pruning matched the clean-model log-loss (0.5818 vs 0.5818 — identical, because the same four features remain), ECE 0.039, fewer parameters, 30/30 data sets improved.

Temperature scaling reached log-loss 0.5894 and ECE 0.052: better than baseline (30/30) but behind pruning, as predicted; accuracy unchanged. The 90 %-correlated case behaves the same way (0.5818 vs 0.6210).

- **New costs:** $O(d^2N)$ pairwise CMI computation, a threshold τ to choose, and possible loss of information when the dropped feature is only partly redundant (here it was an exact copy, so nothing was lost).
- **When the original may remain preferable:** when features really are conditionally independent (pruning gives no gain, as the clean-data check shows); with very small samples, where CMI estimates are noisy and could delete useful features; when the correlated feature carries unique signal (partial correlation) so discarding it hurts; when only the class decision matters, since accuracy was unchanged by the redundancy. When features cannot be removed (e.g. regulatory or measurement reasons), temperature scaling is the cheaper fix.
- **Final decision:** adopt CMI-pruned Naïve Bayes, with temperature scaling as a fallback; both meet the measurable objective on this evidence. Limitation: results come from synthetic binary data with exact or 90 % redundancy; real data should be re-validated.

Appendix – Source Code, Seeds and Configuration

Item	Setting
Python / libraries	Python 3, NumPy, SciPy (linprog for linear-separability check), scikit-learn (CategoricalNB, verification only), matplotlib (plots)
Seeds	Q2 MC 42; Q3 7, 100–129; Q4 42 (sample), 1000–1199 (replicates); Q5 2024 (data), 0–9 and 100–109 (restarts); Q6 11, 5, 9; Q7 7, 100–129
Data partitions	Q3/Q7: 2000 records, random permutation seed = data seed + 1; first 1400 train, last 600 test
Convergence (EM)	$\|\Delta LL\| < 1e\text{-}8$ or 1000 iterations; variance floor 1e-6
AI assistance	This report and code were prepared with AI assistance (Claude); the results come from the code reproduced below and should be understood and explained by the student during the viva.

nb.py – Naïve Bayes from scratch (Q3, Q7)

```
import numpy as np, math
from collections import Counter, defaultdict
class NB:
    """Categorical Naive Bayes from scratch, log-space, add-alpha smoothing."""
    def __init__(self, alpha=1.0): self.alpha=alpha
    def fit(self, X, y, vocab=None):
        X=[tuple(r) for r in X]; self.classes=sorted(set(y)); n=len(y); d=len(X[0])
        self.vocab=vocab or [sorted(set(r[j] for r in X)) for j in range(d)]
        cnt=Counter(y); self.logprior={c:math.log(cnt[c]/n) for c in self.classes}
        self.cpt=[{c:{} for c in self.classes} for _ in range(d)]
        for j in range(d):
            for c in self.classes:
                rows=[r[j] for r,yy in zip(X,y) if yy==c]; k=Counter(rows); V=len(self.vocab[j])
                for v in self.vocab[j]:
                    self.cpt[j][c][v]=(k[v]+self.alpha)/(len(rows)+self.alpha*V) if (len(rows)+self.alpha*V)>0 else 0
        return self
    def log_scores(self, x):
        out={}
        for c in self.classes:
            s=self.logprior[c]
            for j,v in enumerate(x):
                p=self.cpt[j][c].get(v,0.0)
                s+= math.log(p) if p>0 else -math.inf
            out[c]=s
        return out
    def predict_proba(self, X):
        P=[]
        for x in X:
            ls=self.log_scores(x); m=max(ls.values())
            if m==-math.inf: P.append([1/len(self.classes)]*len(self.classes)); continue
            e={c:math.exp(v-m) for c,v in ls.items()}; z=sum(e.values())
            P.append([e[c]/z for c in self.classes])
        return np.array(P)
    def predict(self, X): return [self.classes[i] for i in self.predict_proba(X).argmax(1)]
def logloss(y01,p1):
    p=np.clip(p1,1e-12,1-1e-12); y=np.asarray(y01); return float(-np.mean(y*np.log(p)+(1-y)*np.log(1-p)))
```

q1_q2.py – Q1 and Q2

```
import itertools, math, random, json
R = {}
# ---------- Q1 ----------
pf, sens, fpr = 0.03, 0.95, 0.08
def post(n):
    num = pf*sens**n; den = num + (1-pf)*fpr**n
    return num/den, num, den
R['q1_bayes'] = {n: post(n) for n in (1,2)}
R['q1_prior_sweep'] = {p: (p*sens)/(p*sens+(1-p)*fpr) for p in (0.001,0.03,0.10,0.30,0.50)}
dom = [('High','Normal'),('Heavy','Light'),('High','Low')]
D = [(('High','Heavy','High'),1),(('High','Heavy','Low'),1),(('Normal','Light','Low'),0),
     (('High','Light','High'),0),(('Normal','Heavy','High'),1),(('Normal','Light','High'),0)]
def cov(h,x): return all(a=='?' or (a!='0' and a==b) for a,b in zip(h,x))
def mgen(h1,h2): # h1 more-general-or-equal h2
    return all(a=='?' or a==b for a,b in zip(h1,h2)) or '0' in h2
def fmt(h): return '<'+', '.join('Ø' if a=='0' else a for a in h)+'>'
# Find-S
h = ('0','0','0'); findS=[]
for i,(x,y) in enumerate(D,1):
    if y==1:
        h = tuple(b if a=='0' else (a if a==b else '?') for a,b in zip(h,x))
    findS.append((f'D{i}', 'Yes' if y else 'No', fmt(h)))
R['q1_finds']=findS
```

```
# Candidate elimination
S=[('0','0','0')]; G=[('?','?','?')]; trace=[]
for i,(x,y) in enumerate(D,1):
    if y==1:
        G=[g for g in G if cov(g,x)]
        newS=[]
        for s in S:
            if cov(s,x): newS.append(s)
            else:
                ns=tuple(b if a=='0' else (a if a==b else '?') for a,b in zip(s,x))
                if any(mgen(g,ns) for g in G): newS.append(ns)
        S=newS
    else:
        S=[s for s in S if not cov(s,x)]
        newG=[]
        for g in G:
            if not cov(g,x): newG.append(g)
            else:
                for k in range(3):
                    if g[k]=='?':
                        for v in dom[k]:
                            if v!=x[k]:
                                ng=g[:k]+(v,)+g[k+1:]
                                if any(mgen(ng,s) for s in S): newG.append(ng)
        newG=list(dict.fromkeys(newG))
        G=[g for g in newG if not any(g!=o and mgen(o,g) for o in newG)]
    trace.append((f'D{i}','+' if y else '−',', '.join(fmt(s) for s in S),', '.join(fmt(g) for g in G)))
R['q1_ce']=trace
# version space D1-D3
allH=[tuple(t) for t in itertools.product(*[d+('?',) for d in dom])]+[('0','0','0')]
def consistent(h,data): return all(cov(h,x)==bool(y) for x,y in data)
VS3=[h for h in allH if consistent(h,D[:3])]
VS6=[h for h in allH if consistent(h,D)]
R['q1_vs3']=[fmt(h) for h in VS3]; R['q1_vs6']=[fmt(h) for h in VS6]
R['q1_vs3_post_uniform']=1/len(VS3)
# non-uniform prior
prior={('High','?','?'):0.45,('?','Heavy','?'):0.35,('High','Heavy','?'):0.20}
assert set(prior)==set(VS3)
def lik_noisy(h,data,eps):
    L=1
    for x,y in data: L*= (1-eps) if cov(h,x)==bool(y) else eps
    return L
tab=[]
for h in VS3:
    L=lik_noisy(h,D[:3],0)  # noise free
    tab.append((fmt(h),prior[h],L,prior[h]*L))
Z=sum(t[3] for t in tab)
R['q1_post_nonuniform']=[(a,b,c,d/Z) for a,b,c,d in tab]
# strong prior flip: noisy likelihood on D1-D6, eps=0.2
eps=0.2
def flip(pr):
    out=[]
    for h in VS3:
        L=lik_noisy(h,D,eps); out.append((fmt(h),pr[h],L,pr[h]*L))
    Z=sum(o[3] for o in out); return [(a,b,c,d,d/Z) for a,b,c,d in out]
weak={('High','?','?'):1/3,('?','Heavy','?'):1/3,('High','Heavy','?'):1/3}
strong={('High','?','?'):0.90,('?','Heavy','?'):0.05,('High','Heavy','?'):0.05}
R['q1_flip_weak']=flip(weak); R['q1_flip_strong']=flip(strong)
# new instance vote
xn=('High','Light','Low')
R['q1_newinst']=[(fmt(h),prior[h],'Yes' if cov(h,xn) else 'No') for h in VS3]
yes=sum(prior[h] for h in VS3 if cov(h,xn)); R['q1_vote']=(yes,1-yes)

# ---------- Q2 ----------
c=math.log2(200)
LA=6*12+8*c; LB=25*12+1*c
R['q2_mdl']=dict(log2_200=c,LA=LA,LB=LB,Lh_A=72,LD_A=8*c,Lh_B=300,LD_B=c)
R['q2_cstar']=(8*c-1*c)/(25-6)
cs=[1,2,2.816,3,4,6,12,16]
R['q2_sweep']=[(x,6*x+8*c,25*x+c) for x in cs]
post={'h1':(0.30,'+'),'h2':(0.25,'-'),'h3':(0.20,'-'),'h4':(0.15,'+'),'h5':(0.10,'-')}
vp=sum(p for p,l in post.values() if l=='+'); vm=1-vp
R['q2_vote']=(vp,vm)
R['q2_gibbs']=2*vp*vm; R['q2_bo']=min(vp,vm)
import numpy as np
rng=np.random.default_rng(42); N=200000
names=list(post); probs=np.array([post[k][0] for k in names]); lab=np.array([1 if post[k][1]=='+' else 0 for k in names])
pred=lab[rng.choice(5,size=N,p=probs)]; true=lab[rng.choice(5,size=N,p=probs)]
R['q2_mc']=dict(N=N,seed=42,gibbs_err=float((pred!=true).mean()),bo_err=float((np.where(vp>vm,1,0)!=true).mean()))
json.dump(R,open('r_q1q2.json','w'),indent=1,default=str)
for k,v in R.items(): print(k,v)
```

q3.py – Q3

```
import numpy as np, json, math
from collections import Counter
from nb import NB, logloss
R={}
```

```
data=[('High','Salaried','Good','Yes'),('High','Self-employed','Good','Yes'),('Medium','Salaried','Good','Yes'),
('Medium','Self-employed','Poor','No'),('Low','Salaried','Poor','No'),('Low','Self-employed','Poor','No'),
('Medium','Salaried','Poor','Yes'),('Low','Salaried','Good','Yes'),('High','Self-employed','Poor','No'),('Low','Self-employed','Good','No')]
X=[r[:3] for r in data]; y=[r[3] for r in data]
cnt=Counter(y); R['prior']=dict(cnt)
names=['Income','Employment','Credit']
R['counts']={}
for j,nm in enumerate(names):
    R['counts'][nm]={c:dict(Counter(r[j] for r,yy in zip(X,y) if yy==c)) for c in ('Yes','No')}
m0=NB(alpha=0).fit(X,y)
m1=NB(alpha=1).fit(X,y)
def score_table(m,x):
    # unnormalised scores in prob space and normalised
    sc={}
    for c in m.classes:
        s=math.exp(m.logprior[c])
        for j,v in enumerate(x): s*=m.cpt[j][c].get(v,0.0)
        sc[c]=s
    z=sum(sc.values()); return sc,{c:(s/z if z>0 else float('nan')) for c,s in sc.items()}
tests=[('Low','Self-employed','Poor'),('Medium','Salaried','Good')]
R['manual']={str(t):score_table(m0,t) for t in tests}
# Very High
vocab=[sorted(set(r[0] for r in X))+['Very High'],sorted(set(r[1] for r in X)),sorted(set(r[2] for r in X))]
t=('Very High','Salaried','Good')
mle=NB(alpha=0).fit(X,y,vocab)
R['vh_mle']=score_table(mle,t)
R['vh_alpha']={}
for a in (0.5,1,2):
    ma=NB(alpha=a).fit(X,y,vocab); R['vh_alpha'][a]=(score_table(ma,t), {c:ma.cpt[0][c]['Very High'] for c in ma.classes})
# smoothing also on the two earlier tests
ma=NB(alpha=1).fit(X,y,vocab)
R['smooth_tests']={str(t):score_table(ma,t) for t in tests}
R['cpt_smoothed_income']={c:ma.cpt[0][c] for c in ma.classes}
# library check
from sklearn.naive_bayes import CategoricalNB
from sklearn.preprocessing import OrdinalEncoder
enc=OrdinalEncoder(categories=vocab).fit(X)
Xe=enc.transform(X).astype(int)
sk=CategoricalNB(alpha=1.0,min_categories=[4,2,2]).fit(Xe,y)
tests_all=tests+[t]
mine=ma.predict_proba(tests_all); theirs=sk.predict_proba(enc.transform(tests_all).astype(int))
R['verify']={'classes_mine':ma.classes,'classes_sk':list(sk.classes_),'mine':mine.tolist(),'sk':theirs.tolist(),'maxdiff':float(abs(mine-theirs).max())}
# ---- synthetic experiment ----
def gen(n,seed,corr=None):
    r=np.random.default_rng(seed); y=r.integers(0,2,n)
    def feat(strength): return (r.random(n)<np.where(y==1,strength,1-strength)).astype(int)
    f1=feat(0.70); f2=feat(0.60); f3=feat(0.58); f4=feat(0.55)
    if corr is None: dup=f1.copy()
    else: dup=np.where(r.random(n)<corr,f1,1-f1)
    return y,np.c_[f1,f2,f3,f4,dup]
def run(seed,cols,corr=None,n=2000):
    y,Xs=gen(n,seed,corr); idx=np.random.default_rng(seed+1).permutation(n); tr,te=idx[:1400],idx[1400:]
    m=NB(1.0).fit(Xs[tr][:,cols].tolist(),y[tr].tolist(),vocab=[[0,1]]*len(cols))
    p=m.predict_proba(Xs[te][:,cols].tolist())[:,1]
    return float(((p>0.5).astype(int)==y[te]).mean()), logloss(y[te],p), p, y[te]
res={}
for label,cols,corr in [('without redundant feature',[0,1,2,3],None),('with exact duplicate of x1',[0,1,2,3,4],None),('with 90%-correlated copy of
x1',[0,1,2,3,4],0.9)]:
    a,l,p,yt=run(7,cols,corr); res[label]=(a,l)
R['syn_seed7']=res
agg={}
for label,cols,corr in [('without',[0,1,2,3],None),('dup',[0,1,2,3,4],None),('corr90',[0,1,2,3,4],0.9)]:
    A=[];L=[]
    for s in range(30):
        a,l,_,_=run(100+s,cols,corr); A.append(a); L.append(l)
    agg[label]=(float(np.mean(A)),float(np.std(A)),float(np.mean(L)),float(np.std(L)))
R['syn_30seeds']=agg
json.dump(R,open('r_q3.json','w'),indent=1,default=str)
for k,v in R.items(): print(k,v)
```

q4.py – Q4

```
import itertools, numpy as np, json
R={}
pP,pC=0.10,0.05
pO={(1,1):0.95,(1,0):0.60,(0,1):0.80,(0,0):0.02}
pA={1:0.90,0:0.05}; pS={1:0.70,0:0.01}
def b(p,v): return p if v==1 else 1-p
def joint(P,C,O,A,S): return b(pP,P)*b(pC,C)*b(pO[(P,C)],O)*b(pA[O],A)*b(pS[O],S)
VARS=['P','C','O','A','S']
def enum(query,evidence):
    """P(query=1 | evidence) by enumeration; returns prob, #terms, numerator, denominator"""
    terms=0; num=den=0.0
    for vals in itertools.product([0,1],repeat=5):
        a=dict(zip(VARS,vals))
        if any(a[k]!=v for k,v in evidence.items()): continue
        terms+=1; j=joint(*vals); den+=j
        if a[query]==1: num+=j
```

```
    return num/den,terms,num,den
R['full_joint_sum']=sum(joint(*v) for v in itertools.product([0,1],repeat=5))
R['pO']=enum('O',{}); R['pC_A']=enum('C',{'A':1}); R['pC_AP']=enum('C',{'A':1,'P':1}); R['pC']=pC
R['pP_A']=enum('P',{'A':1}); R['pC_O']=enum('C',{'O':1}); R['pC_OP']=enum('C',{'O':1,'P':1})
R['pA']=enum('A',{}); R['pC_O_P_check']=enum('C',{'O':1,'P':1})
# terms for the four-term manual breakdown
R['manual_O']=[(P,C,b(pP,P)*b(pC,C),pO[(P,C)],b(pP,P)*b(pC,C)*pO[(P,C)]) for P in (1,0) for C in (1,0)]
# P(A=T,C=T) and P(A=T,C=F) breakdown via O
def pO_given(Cv,Pv=None):
    s=0
    for P in (0,1):
        if Pv is not None and P!=Pv: continue
        s+=b(pP,P)*pO[(P,Cv)]
    return s
# independence checks numerically
def marg(**ev):
    return sum(joint(*v) for v in itertools.product([0,1],repeat=5) if all(dict(zip(VARS,v))[k]==x for k,x in ev.items()))
def check_indep(X,Y,Z):
    worst=0
    for z in itertools.product([0,1],repeat=len(Z)):
        ev=dict(zip(Z,z)); pz=marg(**ev)
        if pz==0: continue
        for x in (0,1):
            for y in (0,1):
                pxy=marg(**{X:x,Y:y,**ev})/pz; px=marg(**{X:x,**ev})/pz; py=marg(**{Y:y,**ev})/pz
                worst=max(worst,abs(pxy-px*py))
    return worst
R['indep']={'A⊥S | O':check_indep('A','S',['O']),'P⊥C | Ø':check_indep('P','C',[]),'P⊥C | O':check_indep('P','C',['O']),'P⊥C | A':check_indep('P','C',['A'])}
# sampling
def sample(n,seed):
    r=np.random.default_rng(seed); P=(r.random(n)<pP).astype(int); C=(r.random(n)<pC).astype(int)
    O=np.array([int(r.random()<pO[(p,c)]) for p,c in zip(P,C)])
    A=np.array([int(r.random()<pA[o]) for o in O]); S=np.array([int(r.random()<pS[o]) for o in O])
    return np.c_[P,C,O,A,S]
data=sample(1000,42)
est={}
for n in (50,200,1000):
    d=data[:n]; row={}
    for (p,c),tr in pO.items():
        m=(d[:,0]==p)&(d[:,1]==c); k=int(m.sum())
        row[(p,c)]=(k, float(d[m,2].mean()) if k>0 else None)
    est[n]=row
R['est']={n:{str(k):v for k,v in row.items()} for n,row in est.items()}
R['est_other']={n:dict(P=float(data[:n,0].mean()),C=float(data[:n,1].mean()),
  A_O1=float(data[:n][data[:n,2]==1,3].mean()),A_O0=float(data[:n][data[:n,2]==0,3].mean()),
  S_O1=float(data[:n][data[:n,2]==1,4].mean()),S_O0=float(data[:n][data[:n,2]==0,4].mean())) for n in (50,200,1000)}
# replicated error
rep={}
for n in (50,200,1000):
    errs=[];emp=0;tot=0
    for s in range(200):
        d=sample(n,1000+s)
        e=[]
        for (p,c),tr in pO.items():
            m=(d[:,0]==p)&(d[:,1]==c); tot+=1
            if m.sum()==0: emp+=1; continue
            e.append(abs(d[m,2].mean()-tr))
        errs.append(np.mean(e))
    rep[n]=(float(np.mean(errs)),emp/tot)
R['rep']=rep
json.dump(R,open('r_q4.json','w'),indent=1,default=str)
for k,v in R.items(): print(k,v)
```

q5.py – Q5

```
import numpy as np, json, math
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
R={}
def npdf(x,mu,var): return np.exp(-(x-mu)**2/(2*var))/np.sqrt(2*np.pi*var)
def loglik(x,pi,mu,var):
    return float(np.sum(np.log(np.sum(pi[None,:]*npdf(x[:,None],mu[None,:],var[None,:]),axis=1))))
def estep(x,pi,mu,var):
    w=pi[None,:]*npdf(x[:,None],mu[None,:],var[None,:]); return w/w.sum(1,keepdims=True)
def mstep(x,r,floor=1e-6):
    Nk=r.sum(0); pi=Nk/len(x); mu=(r*x[:,None]).sum(0)/Nk
    var=(r*(x[:,None]-mu[None,:])**2).sum(0)/Nk; return pi,mu,np.maximum(var,floor)
def em(x,pi,mu,var,tol=1e-8,maxit=1000):
    pi,mu,var=map(lambda a:np.array(a,float),(pi,mu,var)); ll=[loglik(x,pi,mu,var)]
    for it in range(maxit):
        r=estep(x,pi,mu,var); pi,mu,var=mstep(x,r); ll.append(loglik(x,pi,mu,var))
        if abs(ll[-1]-ll[-2])<tol: break
    return pi,mu,var,ll
# ---- manual iteration on 7 readings
x7=np.array([1.8,2.5,3.1,5.0,7.6,8.4,9.0])
pi0=np.array([.5,.5]);mu0=np.array([2.,8.]);v0=np.array([1.,1.])
r=estep(x7,pi0,mu0,v0); pi1,mu1,v1=mstep(x7,r)
R['man']=dict(x=x7.tolist(),r=r.tolist(),Nk=r.sum(0).tolist(),pi=pi1.tolist(),mu=mu1.tolist(),var=v1.tolist(),
  ll0=loglik(x7,pi0,mu0,v0),ll1=loglik(x7,pi1,mu1,v1))
```

```
R['man_dens']=(pi0[None,:]*npdf(x7[:,None],mu0[None,:],v0[None,:])).tolist()
# ---- synthetic 500
rng=np.random.default_rng(2024); n=500
true=dict(pi=[0.4,0.6],mu=[2.0,6.0],sd=[1.0,1.5])
z=(rng.random(n)<0.6).astype(int); X=rng.normal(np.array(true['mu'])[z],np.array(true['sd'])[z])
R['true']=true
pi,mu,var,ll=em(X,[.5,.5],[X.min(),X.max()],[X.var(),X.var()])
R['em_main']=dict(pi=pi.tolist(),mu=mu.tolist(),var=var.tolist(),iters=len(ll)-1,ll=ll[-1],min_diff=float(np.min(np.diff(ll))))
fig,ax=plt.subplots(1,2,figsize=(9,3.4))
ax[0].plot(ll,marker='o',ms=3);ax[0].set_xlabel('iteration');ax[0].set_ylabel('log-likelihood');ax[0].set_title('EM log-likelihood (never decreases)')
ax[1].plot(np.arange(1,len(ll)),np.diff(ll),marker='o',ms=3);ax[1].set_yscale('log');ax[1].set_xlabel('iteration');ax[1].set_ylabel('ΔLL');ax[1].set_title('Increase per
iteration')
plt.tight_layout();plt.savefig('fig/q5_ll.png',dpi=150);plt.close()
# ---- 12 inits
rows=[]
for s in range(10):
    r_=np.random.default_rng(s); mu_i=r_.uniform(X.min(),X.max(),2); v_i=r_.uniform(0.5,5,2); p=r_.uniform(0.2,0.8); pi_i=[p,1-p]
    pi_,mu_,v_,l_=em(X,pi_i,mu_i,v_i)
    rows.append((f'rand{s}',mu_i.tolist(),pi_.tolist(),mu_.tolist(),v_.tolist(),l_[-1],len(l_)-1,bool(mu_[0]>mu_[1])))
for name,mu_i,v_i in [('poor-A (both means at 4.0)',[3.9,4.1],[1,1]),('poor-B (means 10, 11)',[10.,11.],[0.5,0.5])]:
    pi_,mu_,v_,l_=em(X,[.5,.5],mu_i,v_i); rows.append((name,mu_i,[.5,.5],mu_.tolist(),v_.tolist(),l_[-1],len(l_)-1,bool(mu_[0]>mu_[1])))
R['inits']=rows
# ---- model selection K=1,2,3
def em_k(x,K,seeds=10):
    best=None
    for s in range(seeds):
        r_=np.random.default_rng(100+s); mu_i=np.sort(r_.choice(x,K,replace=False)); pi_i=np.ones(K)/K; v_i=np.ones(K)*x.var()/K
        res=em(x,pi_i,mu_i,v_i)
        if best is None or res[3][-1]>best[3][-1]: best=res
    return best
ms=[]
for K in (1,2,3):
    p_,m_,v_,l_=em_k(X,K); npar=3*K-1; ll_=l_[-1]
    ms.append((K,npar,ll_,-2*ll_+npar*math.log(n),-2*ll_+2*npar,m_.tolist(),p_.tolist(),v_.tolist()))
R['ms']=ms
# ---- hard vs soft
def kmeans(x,mu,maxit=200):
    mu=np.array(mu,float)
    for _ in range(maxit):
        a=np.abs(x[:,None]-mu[None,:]).argmin(1); new=np.array([x[a==k].mean() for k in range(len(mu))])
        if np.allclose(new,mu): break
        mu=new
    return a,mu
a,mk=kmeans(X,[X.min(),X.max()])
pik=np.array([(a==k).mean() for k in range(2)]); vk=np.array([X[a==k].var() for k in range(2)])
soft=estep(X,pi,mu,var).argmax(1); o=np.argsort(mu); soft_lab=np.where(soft==o[0],0,1)
# true labels: 0 -> component 1 (mean 2) i.e. z==0
acc_soft=float((soft_lab==z).mean()); acc_hard=float((a==z).mean())
R['hard']=dict(mu=mk.tolist(),pi=pik.tolist(),var=vk.tolist(),acc_soft=acc_soft,acc_hard=acc_hard,ll_hard=loglik(X,pik,mk,vk),ll_soft=ll[-1])
# plot histogram + fits
xs=np.linspace(X.min()-1,X.max()+1,400)
fig,ax=plt.subplots(figsize=(6.5,3.4)); ax.hist(X,bins=35,density=True,alpha=.4,label='data')
ax.plot(xs,sum(pi[k]*npdf(xs,mu[k],var[k]) for k in range(2)),label='EM (soft) fit')
ax.plot(xs,sum(pik[k]*npdf(xs,mk[k],vk[k]) for k in range(2)),'--',label='k-means (hard) fit')
ax.legend();ax.set_xlabel('thickness');plt.tight_layout();plt.savefig('fig/q5_fit.png',dpi=150);plt.close()
fig,ax=plt.subplots(figsize=(5,3.2)); ax.plot([m[0] for m in ms],[m[3] for m in ms],'o-',label='BIC'); ax.plot([m[0] for m in ms],[m[4] for m in ms],'s--',label='AIC')
ax.set_xticks([1,2,3]);ax.set_xlabel('components K');ax.set_ylabel('score (lower =
better)');ax.legend();plt.tight_layout();plt.savefig('fig/q5_bic.png',dpi=150);plt.close()
json.dump(R,open('r_q5.json','w'),indent=1,default=str)
for k,v in R.items(): print(k,v)
```

q6.py – Q6

```
import numpy as np, math, json, itertools
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from scipy.optimize import linprog
R={}
def m_pac(n,eps,dl): return (1/eps)*(math.log(3**n+1)+math.log(1/dl))
R['pac_table']=[(e,d,m_pac(10,e,d),math.ceil(m_pac(10,e,d))) for e in (0.1,0.05) for d in (0.05,0.01)]
R['lnH10']=math.log(3**10+1)
ns=np.arange(1,31); fig,ax=plt.subplots(1,3,figsize=(11,3.3))
for e in (0.2,0.1,0.05,0.01): ax[0].plot(ns,[m_pac(n,e,0.05) for n in ns],label=f'ε={e}')
ax[0].set_xlabel('n');ax[0].set_ylabel('m');ax[0].set_title('m vs n (δ=0.05)');ax[0].legend(fontsize=7)
es=np.linspace(0.01,0.3,100)
for n in (5,10,20): ax[1].plot(es,[m_pac(n,e,0.05) for e in es],label=f'n={n}')
ax[1].set_xlabel('ε');ax[1].set_title('m vs ε (δ=0.05)');ax[1].legend(fontsize=7)
ds=np.logspace(-4,-0.3,100)
for n in (5,10,20): ax[2].plot(ds,[m_pac(n,0.1,d) for d in ds],label=f'n={n}')
ax[2].set_xscale('log');ax[2].set_xlabel('δ');ax[2].set_title('m vs δ (ε=0.1)');ax[2].legend(fontsize=7)
plt.tight_layout();plt.savefig('fig/q6_bound.png',dpi=150);plt.close()
base=m_pac(10,0.1,0.05)
R['sens']=dict(eps_half=m_pac(10,0.05,0.05)/base,delta_tenth=m_pac(10,0.1,0.005)/base,n_double=m_pac(20,0.1,0.05)/base)
# --- Find-S simulation
n=10; target={0:1,2:0,4:1} # x1 AND NOT x3 AND x5
tlits=len(target); Ptarget=2.0**-tlits
def sim(m,trials=500,seed=11):
    rng=np.random.default_rng(seed); errs=[]
    for _ in range(trials):
```

```
    X=rng.integers(0,2,(m,n)); pos=np.all([X[:,i]==v for i,v in target.items()],axis=0)
    lits=np.ones((n,2),bool) # lits[i,v]: literal x_i=v still in h
    for x in X[pos]:
      for i in range(n): lits[i,1-x[i]]=False
    L=int(lits.sum()); errs.append(Ptarget-2.0**-L)
  return np.array(errs)
ms=[10,20,30,40,60,80,100,120,140,200,280,400]
simres=[]
for m in ms:
  e=sim(m); simres.append((m,float(e.mean()),float((e<=0.1+1e-12).mean()),float((e<=0.05+1e-12).mean())))
R['sim']=simres
fig,ax=plt.subplots(1,2,figsize=(9,3.3))
ax[0].plot(ms,[s[1] for s in simres],'o-');ax[0].set_xlabel('m');ax[0].set_ylabel('mean true error');ax[0].set_title('Find-S true error vs m')
ax[1].plot(ms,[s[2] for s in simres],'o-',label='P(err ≤ 0.1)');ax[1].plot(ms,[s[3] for s in simres],'s-',label='P(err ≤ 0.05)')
ax[1].axhline(0.95,ls='--',c='gray');ax[1].axvline(140,ls=':',c='r');ax[1].axvline(280,ls=':',c='g');ax[1].legend();ax[1].set_xlabel('m');ax[1].set_title('Fraction of trials meeting ε')
plt.tight_layout();plt.savefig('fig/q6_sim.png',dpi=150);plt.close()
# --- VC checks
def interval_shatters(pts):
  for lab in itertools.product([0,1],repeat=len(pts)):
    P=[p for p,l in zip(pts,lab) if l]; N=[p for p,l in zip(pts,lab) if not l]
    if P and any(min(P)<=q<=max(P) for q in N): return False
  return True
def linsep(pts,lab):
  A=[];b=[]
  for p,l in zip(pts,lab):
    s=1 if l else -1; A.append([-s*p[0],-s*p[1],-s]); b.append(-1)
  r=linprog([0,0,0],A_ub=A,b_ub=b,bounds=[(None,None)]*3); return r.status==0
def lin_shatters(pts): return all(linsep(pts,l) for l in itertools.product([0,1],repeat=len(pts)))
def rect_shatters(pts):
  for lab in itertools.product([0,1],repeat=len(pts)):
    P=[p for p,l in zip(pts,lab) if l]; N=[p for p,l in zip(pts,lab) if not l]
    if P:
      x0,x1=min(p[0] for p in P),max(p[0] for p in P);y0,y1=min(p[1] for p in P),max(p[1] for p in P)
      if any(x0<=q[0]<=x1 and y0<=q[1]<=y1 for q in N): return False
  return True
R['vc']=dict(interval2=interval_shatters([1,2]),interval3=interval_shatters([1,2,3]),
 lin3=lin_shatters([(0,0),(1,0),(0,1)]),lin4_xor=lin_shatters([(0,0),(1,1),(1,0),(0,1)]),lin4_other=lin_shatters([(0,0),(2,0),(1,1),(1,0.3)]),
 rect4=rect_shatters([(0,1),(0,-1),(1,0),(-1,0)]),rect5=rect_shatters([(0,1),(0,-1),(1,0),(-1,0),(0,0)]))
def m_vc(d,eps=0.1,dl=0.05): return (1/eps)*(4*math.log2(2/dl)+8*d*math.log2(13/eps))
R['vcbound']=[(nm,d,m_vc(d),math.ceil(m_vc(d))) for nm,d in (('Intervals on R',2),('Linear separators in R²',3),('Axis-aligned rectangles in R²',4))]
# --- Halving: H = monotone conjunctions over 6 vars (64 hypotheses)
nv=6; Hs=[frozenset(s) for r in range(nv+1) for s in itertools.combinations(range(nv),r)]
assert len(Hs)==64
def h_eval(h,x): return all(x[i] for i in h)
def halving(target,seq):
  V=list(Hs); mist=0
  for x in seq:
    votes=sum(h_eval(h,x) for h in V); pred=votes*2>len(V) if votes*2!=len(V) else False
    y=h_eval(target,x)
    if pred!=y: mist+=1
    V=[h for h in V if h_eval(h,x)==y]
  return mist,len(V)
rng=np.random.default_rng(5); mm=[]
for t in range(2000):
  tg=Hs[rng.integers(64)]; seq=rng.integers(0,2,(60,nv)); mm.append(halving(tg,seq)[0])
R['halving']=dict(max=int(max(mm)),mean=float(np.mean(mm)),bound=math.log2(64),dist={int(k):int(v) for k,v in zip(*np.unique(mm,return_counts=True))})
# worst-case-ish sequence for halving: biased examples with p=0.5 vs 0.9
mm2=[]
for t in range(2000):
  tg=Hs[rng.integers(64)]; seq=(rng.random((60,nv))<0.8).astype(int); mm2.append(halving(tg,seq)[0])
R['halving2']=dict(max=int(max(mm2)),mean=float(np.mean(mm2)))
# --- online conjunction learner
def online(target,seq,nn):
  lits=np.ones((nn,2),bool); mist=0
  for x in seq:
    pred=not any(lits[i,1-x[i]] for i in range(nn)); y=all(x[i]==v for i,v in target.items())
    if pred!=y: mist+=1
    if y:
      for i in range(nn): lits[i,1-x[i]]=False
  return mist
res={}
for nn in (5,10,20):
  rng=np.random.default_rng(9); mx=0; tot=[]
  for t in range(5000):
    k=rng.integers(0,nn+1); idx=rng.choice(nn,k,replace=False); tg={int(i):int(rng.integers(0,2)) for i in idx}
    # bias examples toward positives so mistakes can occur
    seq=[]
    for _ in range(200):
      x=rng.integers(0,2,nn)
      if rng.random()<0.5:
        for i,v in tg.items(): x[i]=v
      seq.append(x)
    mst=online(tg,seq,nn); tot.append(mst); mx=max(mx,mst)
  # adversarial: empty target, x0 = all ones, then flip one bit at a time
  adv=[np.ones(nn,int)]
  for i in range(nn):
    x=np.ones(nn,int); x[i]=0; adv.append(x)
  res[nn]=dict(max=mx,mean=float(np.mean(tot)),adv=online({},adv,nn),bound=nn+1)
R['online']=res
# --- Opt
```

```
from functools import lru_cache
def Opt(C,X):
    C=frozenset(C)
    @lru_cache(None)
    def M(V):
        if len(V)<=1: return 0
        best=0
        for x in X:
            V1=frozenset(c for c in V if x in c); V0=V-V1
            if not V1 or not V0: continue
            # learner picks prediction b minimising worst-case; adversary then picks label
            cand=[]
            for Vb,Vn in ((V1,V0),(V0,V1)):
                cand.append(max(M(Vb), 1+M(Vn)))
            best=max(best,min(cand))
        return best
    return M(C)
def VC(C,X):
    C=list(C); best=0
    for k in range(1,len(X)+1):
        ok=False
        for S in itertools.combinations(X,k):
            if len({tuple(x in c for x in S) for c in C})==2**k: ok=True;break
        if ok: best=k
        else: break
    return best
X4=[1,2,3,4]; thr=[frozenset(x for x in X4 if x>=k) for k in range(1,6)]
X3=[1,2,3]; allsub=[frozenset(s) for r in range(4) for s in itertools.combinations(X3,r)]
X3b=[1,2,3]; sing=[frozenset([x]) for x in X3]+[frozenset()]
R['opt']=[('Thresholds on 4 ordered points (5 concepts)',VC(thr,X4),Opt(thr,X4),math.log2(5)),
          ('All subsets of 3 points (8 concepts)',VC(allsub,X3),Opt(allsub,X3),math.log2(8)),
          ('Singletons + empty on 3 points (4 concepts)',VC(sing,X3b),Opt(sing,X3b),math.log2(4))]
json.dump(R,open('r_q6.json','w'),indent=1,default=str)
for k,v in R.items(): print(k,v)
```

q7.py – Q7

```
import numpy as np, json, time, math
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from scipy.optimize import minimize_scalar
from nb import NB, logloss
R={}
def gen(n,seed,corr=None):
    r=np.random.default_rng(seed); y=r.integers(0,2,n)
    def feat(s): return (r.random(n)<np.where(y==1,s,1-s)).astype(int)
    f1=feat(0.70); f2=feat(0.60); f3=feat(0.58); f4=feat(0.55)
    dup=f1.copy() if corr is None else np.where(r.random(n)<corr,f1,1-f1)
    return y,np.c_[f1,f2,f3,f4,dup]
def mi(a,b,w=None):
    N=len(a); out=0
    for u in (0,1):
        for v in (0,1):
            p=np.mean((a==u)&(b==v));
            if p>0: out+=p*math.log(p/(np.mean(a==u)*np.mean(b==v)))
    return out
def cmi(a,b,y):
    return sum(np.mean(y==c)*mi(a[y==c],b[y==c]) for c in (0,1))
def prune(Xtr,ytr,tau=0.05):
    d=Xtr.shape[1]; keep=list(range(d)); rel=[mi(Xtr[:,j],ytr) for j in range(d)]
    changed=True
    while changed:
        changed=False
        pairs=sorted(((cmi(Xtr[:,i],Xtr[:,j],ytr),i,j) for a,i in enumerate(keep) for j in keep[a+1:]),reverse=True)
        if pairs and pairs[0][0]>tau:
            _,i,j=pairs[0]; drop=j if rel[j]<=rel[i] else i; keep.remove(drop); changed=True
    return keep
def ece(p,y,bins=10):
    idx=np.minimum((p*bins).astype(int),bins-1); e=0
    for b in range(bins):
        m=idx==b
        if m.any(): e+=m.mean()*abs(p[m].mean()-y[m].mean())
    return float(e)
def evalp(p,y): return dict(acc=float(((p>0.5)==y).mean()),ll=logloss(y,p),ece=ece(p,y))
def logit(p): p=np.clip(p,1e-12,1-1e-12); return np.log(p/(1-p))
def run(seed,corr,detail=False):
    y,Xs=gen(2000,seed,corr); idx=np.random.default_rng(seed+1).permutation(2000); tr,te=idx[:1400],idx[1400:]
    Xtr,ytr,Xte,yte=Xs[tr],y[tr],Xs[te],y[te]
    out={}
    t=time.perf_counter(); m=NB(1.0).fit(Xtr.tolist(),ytr.tolist(),vocab=[[0,1]]*5); p0=m.predict_proba(Xte.tolist())[:,1]; t0=time.perf_counter()-t
    out['baseline NB (all 5)']=evalp(p0,yte)|dict(feats=5,params=1+5*2,t=t0)
    t=time.perf_counter(); keep=prune(Xtr,ytr)
    m2=NB(1.0).fit(Xtr[:,keep].tolist(),ytr.tolist(),vocab=[[0,1]]*len(keep)); p1=m2.predict_proba(Xte[:,keep].tolist())[:,1]; t1=time.perf_counter()-t
    out['CMI-pruned NB']=evalp(p1,yte)|dict(feats=len(keep),params=1+2*len(keep),t=t1,keep=keep)
    # temperature scaling fitted on training predictions
    ptr=m.predict_proba(Xtr.tolist())[:,1]; z=logit(ptr)
    T=minimize_scalar(lambda T: logloss(ytr,1/(1+np.exp(-z/T))),bounds=(0.2,10),method='bounded').x
    p2=1/(1+np.exp(-logit(p0)/T)); out['NB + temperature scaling']=evalp(p2,yte)|dict(feats=5,params=12,T=float(T))
    if detail: out['_p']=(p0,p1,p2,yte)
```

```
    return out
R['seed7']={}
for lab,corr in (('exact duplicate',None),('90% correlated',0.9)):
    o=run(7,corr,True); R['seed7'][lab]={k:v for k,v in o.items() if k!='_p'}
    if corr is None: P=o['_p']
agg={}
for lab,corr in (('exact duplicate',None),('90% correlated',0.9)):
    runs=[run(100+s,corr) for s in range(30)]; agg[lab]={}
    for k in runs[0]:
        agg[lab][k]={m:(float(np.mean([r[k][m] for r in runs])),float(np.std([r[k][m] for r in runs]))) for m in ('acc','ll','ece','feats','params')}
        agg[lab][k]['wins_vs_base']=int(sum(r[k]['ll']<r['baseline NB (all 5)']['ll'] for r in runs))
R['agg']=agg
# clean data check (no redundant feature present): prune should keep all 4
y,Xs=gen(2000,100); keeps=[]
for s in range(30):
    y,Xs=gen(2000,100+s); keeps.append(len(prune(Xs[:1400,:4],y[:1400])))
R['clean_keep']=float(np.mean(keeps))
# baseline numbers from Q3 clean
# plots
fig,ax=plt.subplots(1,2,figsize=(9,3.4))
labs=['baseline NB (all 5)','CMI-pruned NB','NB + temperature scaling']; short=['Baseline','Pruned','Temp-scaled']
x=np.arange(3);w=0.35
for i,(lab,col) in enumerate((('exact duplicate','C0'),('90% correlated','C1'))):
    ax[0].bar(x+i*w,[agg[lab][l]['ll'][0] for l in labs],w,yerr=[agg[lab][l]['ll'][1] for l in labs],label=lab,color=col,capsize=3)
ax[0].set_xticks(x+w/2);ax[0].set_xticklabels(short);ax[0].set_ylabel('test log-loss (30 seeds)');ax[0].set_ylim(0.5,0.7);ax[0].legend(fontsize=8)
p0,p1,p2,yte=P
for p,l in ((p0,'Baseline'),(p1,'Pruned'),(p2,'Temp-scaled')):
    bins=np.minimum((p*8).astype(int),7); xs=[];ys=[]
    for b in range(8):
        m=bins==b
        if m.sum()>10: xs.append(p[m].mean()); ys.append(yte[m].mean())
    ax[1].plot(xs,ys,'o-',label=l)
ax[1].plot([0,1],[0,1],'k--',lw=.8);ax[1].set_xlabel('predicted P(y=1)');ax[1].set_ylabel('observed frequency');ax[1].set_title('Reliability (seed 7, exact
dup.)');ax[1].legend(fontsize=8)
plt.tight_layout();plt.savefig('fig/q7_results.png',dpi=150);plt.close()
json.dump(R,open('r_q7.json','w'),indent=1,default=str)
print(json.dumps(R,indent=1,default=str))
```